

MÉMOIRE

Sous thème

Réseaux de neurones convolutifs

Master 1 Ingénierie Mathématique et Data Science (IMDS)

Faculté des Sciences et Techniques – Université Haute-Alsace

Encadré par : Monsieur Makhoul Abdenacer

Réalisé par :

- Assoulaimani Naima
- Darid Khadija

Année universitaire : 2025 – 2026

Remerciements

Nous tenons à exprimer nos plus sincères remerciements à toutes les personnes qui ont contribué à la réalisation de ce rapport et au succès de cette année d'études enrichissante.

Nous souhaitons adresser notre profonde gratitude à notre encadrant, **Monsieur MAKHLOUF Abdenacer**, pour avoir accepté de nous encadrer et de nous guider dans ce projet. Nous le remercions chaleureusement pour sa confiance, sa bienveillance et pour la pertinence de ses directives, qui nous ont permis de mener à bien notre travail sur les réseaux de neurones convolutifs.

Nos remerciements s'étendent également à tous nos enseignants, dont l'expertise et le dévouement ont grandement enrichi notre expérience académique.

Enfin, nous tenons à exprimer notre reconnaissance envers les membres du jury qui ont accepté d'évaluer ce travail, ainsi qu'à nos proches pour leurs encouragements quotidiens.

Table des matières

Remerciements	i
Introduction	1
1 Le principe des réseaux de neurones artificiels	2
1.1 Invention des premiers neurones artificiels	2
1.1.1 Warren McCulloch et Walter Pitts	2
1.1.2 Les neurones biologiques	3
1.1.3 Le modèle mathématique	3
1.2 Le perceptron	4
1.2.1 Le modèle linéaire	4
1.2.2 La fonction d'activation	5
1.2.3 La fonction coût	6
1.2.4 La descente de gradient	7
1.2.5 La vectorisation des équations	8
1.3 Un réseau de neurones	9
1.3.1 Les couches de neurones	9
1.3.2 La propagation avant	11
1.3.3 La rétropropagation	12
2 Réseaux de neurones convolutifs	14
2.1 Introduction	15
2.2 Limites des réseaux entièrement connectés pour les images	15
2.2.1 Rappel sur le perceptron multicouche	15
2.2.2 Explosion du nombre de paramètres	16
2.2.3 Perte de la structure spatiale	17
2.3 Intuition et architecture générale d'un CNN	17
2.3.1 Principe général	17
2.3.2 Rôle des couches principales	18
2.4 A priori sur les images et intérêt des CNN	18
2.5 Définition mathématique de la convolution	19
2.5.1 Convolution continue	19

2.5.2	Convolution discrète unidimensionnelle	20
2.6	Convolution en deux dimensions	21
2.6.1	Principe général	21
2.6.2	Exemple avec un filtre 3×3	22
2.7	Corrélation croisée et interprétation de la convolution	23
2.7.1	Différence entre convolution et corrélation croisée	23
2.7.2	Interprétation comme produit scalaire local	24
2.8	Padding et stride	25
2.8.1	Padding	25
2.8.2	Stride	26
2.9	Filtres classiques pour le traitement d'image	26
2.9.1	Filtre moyenneur	26
2.9.2	Filtre rehausseur	27
2.9.3	Filtre laplacien	27
2.9.4	Filtres de Sobel	28
2.10	Fonction d'activation ReLU	29
2.11	Couche convolutive complète	29
2.11.1	Entrée, sortie et paramètres de la couche	29
2.11.2	Calcul d'une carte d'activation	31
2.11.3	Nombre de paramètres	31
2.11.4	Convolution sur un tenseur	32
2.11.5	Exemple numérique d'une couche convolutive	33
2.12	Pooling	34
2.12.1	Définition générale	34
2.12.2	Max pooling et average pooling	34
2.13	Équivariance, invariance et rôle du pooling	35
2.14	Rétropropagation dans une couche convolutive	35
2.15	Rétropropagation dans une couche de pooling	36
2.15.1	Gradient du max pooling	36
2.15.2	Gradient de l'average pooling	37
2.16	Non-linéarité dans une couche convolutive	37
2.17	Hierarchie de convolutions et champ réceptif	38
2.17.1	Principe de la hiérarchie de convolutions	38
2.17.2	Exemple : détection de contours par deux couches	39

2.17.3	Champ réceptif et échantillonnage des cartes d'activation	41
2.18	Couches entièrement connectées et classification	43
2.19	Exemple d'architecture : LeNet-5	44
2.19.1	Principe général de l'architecture	45
2.19.2	Première convolution et premier sous-échantillonnage	45
2.19.3	Deuxième convolution et deuxième sous-échantillonnage	46
2.19.4	Troisième convolution et passage vers la classification	46
2.19.5	Résumé de LeNet-5	47
2.20	Synthèse du fonctionnement d'un CNN	47
2.21	Conclusion	48
3	Application d'un CNN sur la base MNIST	49
3.1	Introduction	49
3.2	Environnement de travail	49
3.3	Chargement et visualisation de la base MNIST	49
3.4	Prétraitement des données	50
3.5	Architecture du modèle CNN	51
3.6	Compilation et entraînement	52
3.7	Évaluation et analyse des résultats	54
3.8	Sauvegarde du modèle	55
3.9	Conclusion	55

Table des figures

1.1	Réseau de neurones artificiels	2
1.2	Warren McCulloch et Walter Pitts	2
1.3	Neurone biologique	3
1.4	Le modèle mathématique du neurone artificiel	4
1.5	Plantes toxiques et non toxiques	4
1.6	La représentation graphique des plantes toxiques et non toxiques	5
1.7	Le modèle linéaire	5
1.8	La fonction sigmoïde	6
1.9	La fonction coût	6
1.10	La descente de gradient	7
1.11	Réseau de neurones à une couche	9
1.12	Organisation des couches dans un réseau de neurones.	10
1.13	Ajout progressif de neurones et de couches.	10
1.14	Réseau de neurones à plusieurs couches	10
1.15	Exemple d'un réseau de neurones	11
1.16	Exemple d'un réseau de neurones	13
2.1	Architecture générale d'un réseau de neurones convolutif pour la classification d'images.	15
2.2	Transformation d'une image en vecteur pour un réseau entièrement connecté.	17
2.3	Architecture générale d'un réseau de neurones convolutif : extraction de caractéristiques puis classification.	18
2.4	Les deux opérations de base de la phase d'extraction des caractéristiques. .	18
2.5	Stabilité des représentations face aux petites déformations et translations. .	19
2.6	Exemple de convolution discrète 1D : le filtre glisse sur le signal d'entrée et produit une valeur du signal de sortie.	21
2.7	Principe de la convolution 2D : un filtre est appliqué localement sur une région de l'image pour produire une valeur de la carte de sortie.	21
2.8	Différence entre corrélation croisée et convolution stricte : dans la convolution, le filtre est retourné avant application.	23
2.9	Interprétation de la corrélation croisée comme produit scalaire local entre une région de l'image et un filtre.	24

2.10	Zero-padding : ajout de pixels nuls autour de l'image pour traiter les bords.	25
2.11	Illustration d'une convolution avec un filtre 3×3 , un padding de 1 et un stride de 2.	26
2.12	Illustration de l'effet d'un filtre moyenneur : l'image est lissée et les variations locales sont adoucies.	27
2.13	Illustration de l'effet d'un filtre rehausseur : les contours et les détails de l'image sont accentués.	27
2.14	Illustration de l'effet d'un filtre laplacien : les contours sont extraits de l'image.	28
2.15	Résultats du filtre de Sobel : de gauche à droite, image source, réponse selon G_x , réponse selon G_y et image finale des contours.	28
2.16	La fonction ReLU conserve les valeurs positives et annule les valeurs négatives.	29
2.17	Un filtre convolutif parcourt localement l'image en largeur et en hauteur, tout en prenant en compte la profondeur du tenseur.	30
2.18	Exemple de convolution sur une image à trois canaux : les réponses obtenues sur chaque canal sont additionnées pour former une carte de sortie.	31
2.19	Exemple numérique d'une convolution 3D : deux filtres $3 \times 3 \times 3$ appliqués à un volume d'entrée produisent deux cartes d'activation.	33
2.20	Comparaison entre max pooling et average pooling.	34
2.21	Échantillonnage des cartes filtrées par pooling spatial.	35
2.22	Une couche convolutive transforme un tenseur d'entrée en tenseur de sortie grâce à deux étapes : une convolution affine suivie d'une activation non linéaire.	37
2.23	Principe de la hiérarchie de convolutions : les couches successives combinent progressivement des informations locales pour construire des représentations plus complexes.	38
2.24	Première couche de l'exemple : application de deux filtres de Sobel sur une image en niveaux de gris pour obtenir deux cartes de contours.	40
2.25	Deuxième couche de l'exemple : combinaison des deux cartes de contours pour produire une sortie plus complète.	41
2.26	Illustration du champ réceptif : une activation profonde dépend indirectement d'une zone plus large de l'image d'entrée.	42
2.27	Échantillonnage des cartes d'activation : le pooling réduit la largeur et la hauteur, tout en conservant le nombre de canaux.	43
2.28	Flattening : transformation des cartes d'activation en vecteur avant les couches entièrement connectées.	44
2.29	Architecture générale de LeNet-5 : succession de convolutions, de sous-échantillonnages et de couches entièrement connectées.	45

3.1	Extrait du notebook : chargement de la base MNIST	50
3.2	Exemples d'images de la base MNIST	50
3.3	Extrait du notebook : prétraitement des images et des labels	51
3.4	Extrait du notebook : construction du modèle CNN	51
3.5	Extrait du notebook : compilation et entraînement du modèle	52
3.6	Évolution de la précision pendant l'entraînement	53
3.7	Évolution de la fonction de perte pendant l'entraînement	53
3.8	Extrait du notebook : évaluation du modèle et prédiction des classes	54
3.9	Matrice de confusion du modèle CNN sur MNIST	54
3.10	Exemples de prédictions réalisées par le modèle	55
3.11	Extrait du notebook : sauvegarde du modèle entraîné	55

Introduction

Les réseaux de neurones artificiels occupent aujourd'hui une place importante dans le domaine de l'intelligence artificielle et de l'apprentissage automatique. Leur principe général consiste à apprendre automatiquement une relation entre des données d'entrée et une sortie attendue, à partir d'exemples. Cette capacité d'apprentissage permet de traiter des problèmes variés comme la classification, la reconnaissance de formes, l'analyse d'images ou encore la détection d'objets.

Dans le cadre de ce projet, nous nous intéressons plus particulièrement à l'utilisation des réseaux de neurones dans le domaine de la vision par ordinateur. Les images sont des données particulières, car elles possèdent une structure spatiale : les pixels voisins sont liés entre eux et forment des contours, des textures, des formes et des objets. Pour cette raison, les réseaux de neurones convolutifs, ou CNN, sont devenus des architectures essentielles pour l'analyse automatique des images.

L'objectif principal de ce travail est donc de présenter les bases nécessaires à la compréhension des CNN, puis d'expliquer leur fonctionnement dans un contexte de traitement d'images. Le premier chapitre constitue une introduction générale aux réseaux de neurones artificiels. Il présente le neurone artificiel, le perceptron, les fonctions d'activation, la fonction coût, la descente de gradient, la propagation avant et la rétropropagation.

Le deuxième chapitre est consacré aux réseaux de neurones convolutifs. Il détaille les opérations fondamentales utilisées dans ces architectures, notamment la convolution, les filtres, le padding, le pooling, les cartes d'activation, les couches entièrement connectées et la notion de champ réceptif. Ces éléments permettent de comprendre comment un CNN extrait progressivement des caractéristiques d'une image pour réaliser une tâche de classification ou de reconnaissance visuelle.

Enfin, la dernière partie du projet sera consacrée à une application des réseaux de neurones convolutifs. Elle permettra de mettre en pratique les notions théoriques étudiées précédemment et de montrer comment un CNN peut être utilisé sur des images réelles.

Chapitre 1

Le principe des réseaux de neurones artificiels

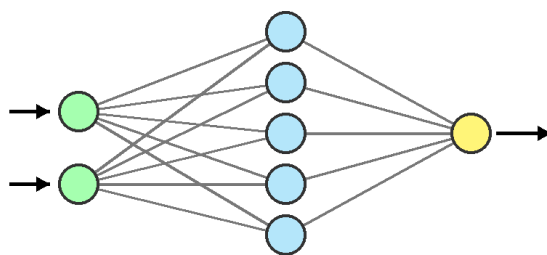


Figure 1.1 – Réseau de neurones artificiels

1.1 Invention des premiers neurones artificiels

1.1.1 Warren McCulloch et Walter Pitts

Les premiers modèles de neurones artificiels ont été proposés en 1943 par Warren McCulloch et Walter Pitts. Leur objectif était de formaliser mathématiquement le comportement élémentaire d'un neurone biologique. Dans leur article intitulé *A logical calculus of the ideas immanent in nervous activity*, ils montrent qu'un neurone peut être représenté comme une unité logique capable de recevoir plusieurs signaux et de produire une sortie binaire.



Figure 1.2 – Warren McCulloch et Walter Pitts

1.1.2 Les neurones biologiques

En biologie, un neurone est une cellule spécialisée dans la transmission d'informations. Il est principalement composé de dendrites, d'un corps cellulaire et d'un axone. Les dendrites reçoivent des signaux provenant d'autres neurones. Lorsque la somme de ces signaux dépasse un certain seuil, le neurone s'active et transmet un signal électrique le long de l'axone vers d'autres neurones.

Cette logique d'activation par seuil a fortement inspiré la construction des premiers neurones artificiels. L'idée centrale est de considérer qu'un neurone artificiel reçoit plusieurs valeurs d'entrée, les combine, puis décide de s'activer ou non.

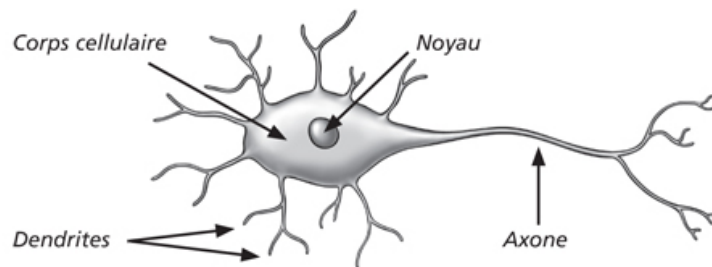


Figure 1.3 – Neurone biologique

1.1.3 Le modèle mathématique

Un neurone artificiel peut être décrit comme une fonction recevant plusieurs entrées $x_1, x_2, \dots, x_N$ et produisant une sortie y . Le calcul se fait en deux étapes principales.

La première étape est l'agrégation. Chaque entrée est multipliée par un poids ω_i , puis l'ensemble est additionné :

$$f = \omega_1 x_1 + \omega_2 x_2 + \dots + \omega_N x_N. \quad (1.1)$$

La seconde étape est l'activation. Le neurone compare la valeur calculée à un seuil. Dans le cas le plus simple, ce seuil est égal à zéro :

$$y = \begin{cases} 1, & \text{si } f \geq 0, \\ 0, & \text{sinon.} \end{cases} \quad (1.2)$$

Ainsi, un neurone artificiel peut être vu comme une unité de décision élémentaire. En reliant plusieurs neurones entre eux, il devient possible de construire des modèles capables de représenter des fonctions plus complexes.

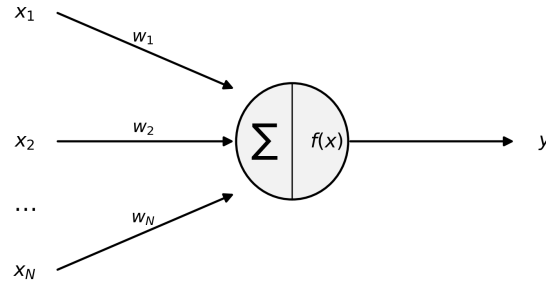


Figure 1.4 – Le modèle mathématique du neurone artificiel

1.2 Le perceptron

1.2.1 Le modèle linéaire

Le perceptron est l’une des premières formes de réseau de neurones. Il s’agit d’un modèle de classification binaire permettant de séparer deux classes lorsque celles-ci sont linéairement séparables.

Considérons par exemple un problème de classification de plantes. Certaines plantes sont toxiques et sont notées $y = 1$, tandis que les plantes non toxiques sont notées $y = 0$.



Figure 1.5 – Plantes toxiques et non toxiques

Pour chaque plante, on mesure deux caractéristiques : la largeur des feuilles x_1 et la longueur des feuilles x_2 .

Table 1.1 – Exemple de données pour la classification de plantes

y	x_1	x_2
1	0.5	2.0
1	1.1	2.1
1	0.7	2.6
0	2.0	1.0
0	2.5	0.7
0	2.2	0.3

Si les deux groupes de points peuvent être séparés par une droite, cette droite est appelée frontière de décision. Le rôle du perceptron est alors de déterminer une équation permettant

de séparer au mieux les deux classes.

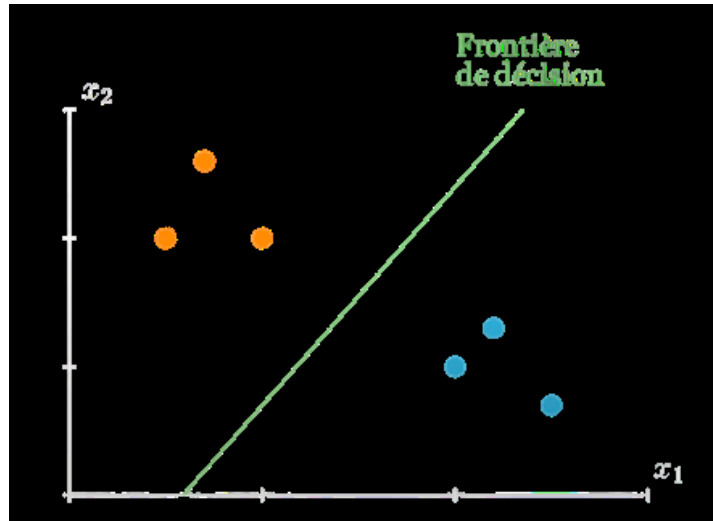


Figure 1.6 – La représentation graphique des plantes toxiques et non toxiques

Le modèle linéaire associé au perceptron s'écrit :

$$z(x_1, x_2) = \omega_1 x_1 + \omega_2 x_2 + b, \quad (1.3)$$

où ω_1 et ω_2 sont les poids associés aux variables d'entrée, et b est le biais. Le biais permet de déplacer la frontière de décision afin d'améliorer la séparation des classes.

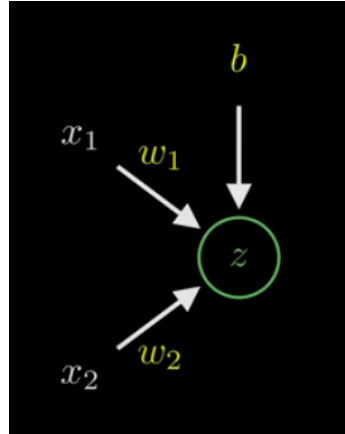


Figure 1.7 – Le modèle linéaire

La prédiction du perceptron est donnée par :

$$\hat{y} = \begin{cases} 0, & \text{si } z \leq 0, \\ 1, & \text{si } z > 0. \end{cases} \quad (1.4)$$

1.2.2 La fonction d'activation

La sortie linéaire z permet de prendre une décision, mais elle ne fournit pas directement une probabilité. Pour obtenir une sortie comprise entre 0 et 1, on utilise une fonction

d'activation. Dans un problème de classification binaire, la fonction sigmoïde est souvent utilisée :

$$a(z) = \sigma(z) = \frac{1}{1 + e^{-z}}. \quad (1.5)$$

Cette fonction transforme la valeur z en une probabilité. Par exemple, une valeur $a(z) = 0.8$ signifie que le modèle estime que l'observation a 80% de chances d'appartenir à la classe 1. Inversement, une valeur $a(z) = 0.1$ indique une faible probabilité d'appartenance à cette classe.

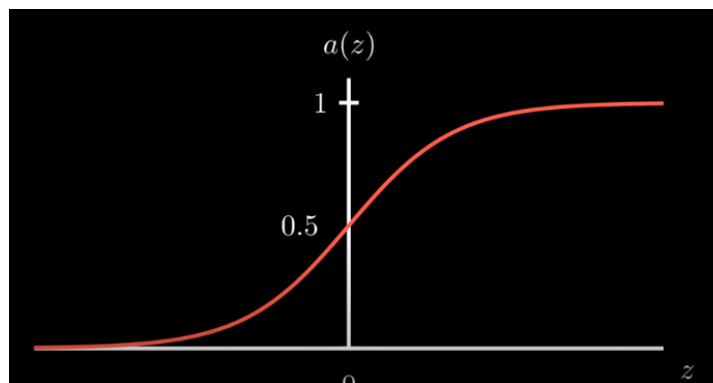


Figure 1.8 – La fonction sigmoïde

1.2.3 La fonction coût

Pour entraîner un modèle, il est nécessaire de mesurer l'écart entre les prédictions du modèle et les vraies valeurs observées. Cette mesure est appelée fonction coût ou fonction de perte.

Dans le cas d'une classification binaire, la fonction coût la plus utilisée est la Log Loss, définie par :

$$L = -\frac{1}{m} \sum_{i=1}^m [y_i \log(a_i) + (1 - y_i) \log(1 - a_i)], \quad (1.6)$$

où m est le nombre d'observations, y_i la vraie classe de l'observation i , et a_i la probabilité prédite par le modèle.

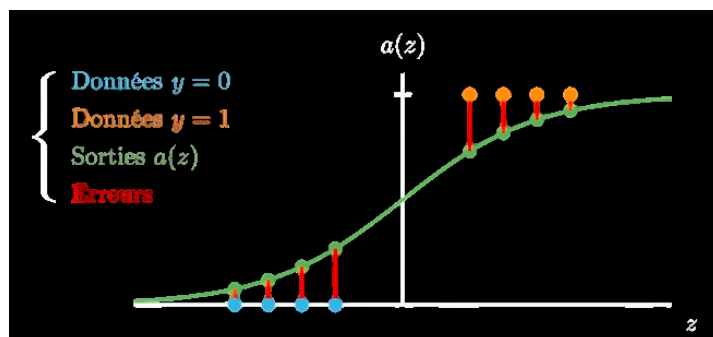


Figure 1.9 – La fonction coût

Cette fonction pénalise fortement les prédictions confiantes mais incorrectes. Si la vraie classe est 1, le modèle doit produire une probabilité proche de 1. Si la vraie classe est 0, il doit produire une probabilité proche de 0.

La fonction Log Loss peut être reliée au principe de vraisemblance. Pour une variable de Bernoulli, on a :

$$P(Y = y) = a(z)^y(1 - a(z))^{1-y}. \quad (1.7)$$

Pour l'ensemble des observations, la vraisemblance est le produit :

$$\mathcal{L} = \prod_{i=1}^m a_i^{y_i}(1 - a_i)^{1-y_i}. \quad (1.8)$$

Comme le produit de nombreuses probabilités peut devenir très petit, on utilise le logarithme :

$$\log(\mathcal{L}) = \sum_{i=1}^m [y_i \log(a_i) + (1 - y_i) \log(1 - a_i)]. \quad (1.9)$$

Maximiser cette log-vraisemblance revient à minimiser son opposée normalisée, ce qui donne la fonction Log Loss précédente.

1.2.4 La descente de gradient

La descente de gradient est un algorithme d'optimisation permettant d'ajuster les paramètres du modèle afin de minimiser la fonction coût. Les paramètres ω et b sont mis à jour de manière itérative.

La règle de mise à jour d'un poids est :

$$\omega_{t+1} = \omega_t - \alpha \frac{\partial L}{\partial \omega_t}, \quad (1.10)$$

où α est le pas d'apprentissage. Si le gradient est positif, le poids diminue ; s'il est négatif, le poids augmente. En répétant cette opération, le modèle se rapproche progressivement d'un minimum de la fonction coût.

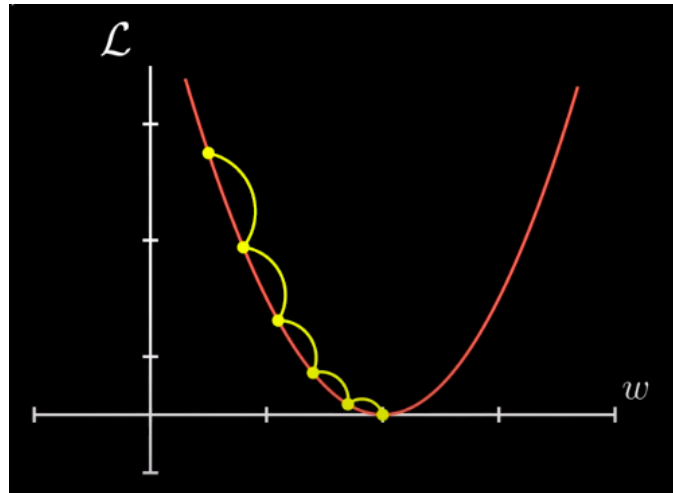


Figure 1.10 – La descente de gradient

Les principales équations du perceptron sont les suivantes :

$$z = \omega_1 x_1 + \omega_2 x_2 + \dots + \omega_n x_n + b, \quad (1.11)$$

$$a = \frac{1}{1 + e^{-z}}, \quad (1.12)$$

$$L = -\frac{1}{m} \sum_{i=1}^m [y_i \log(a_i) + (1 - y_i) \log(1 - a_i)]. \quad (1.13)$$

Pour calculer le gradient, on applique la règle de dérivation en chaîne :

$$\frac{\partial L}{\partial \omega_i} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial \omega_i}. \quad (1.14)$$

On obtient finalement :

$$\frac{\partial L}{\partial \omega_i} = \frac{1}{m} \sum_{i=1}^m (a_i - y_i) x_i, \quad \frac{\partial L}{\partial b} = \frac{1}{m} \sum_{i=1}^m (a_i - y_i). \quad (1.15)$$

Ces dérivées permettent de corriger les paramètres du modèle jusqu'à obtenir une erreur suffisamment faible.

1.2.5 La vectorisation des équations

Dans la pratique, les calculs ne sont pas réalisés observation par observation. Ils sont vectorisés afin d'améliorer l'efficacité numérique. On regroupe les données dans une matrice X :

$$X = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_n^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_n^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(m)} & x_2^{(m)} & \dots & x_n^{(m)} \end{bmatrix} \in \mathbb{R}^{m \times n}. \quad (1.16)$$

Le vecteur des sorties réelles est :

$$Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}. \quad (1.17)$$

La forme matricielle du modèle linéaire est alors :

$$Z = XW + b, \quad (1.18)$$

où $W = [\omega_1, \omega_2, \dots, \omega_n]^T$. La fonction d'activation appliquée à l'ensemble des observations s'écrit :

$$A = \sigma(Z). \quad (1.19)$$

Le gradient vectorisé par rapport aux poids est :

$$\frac{\partial L}{\partial W} = \frac{1}{m} X^T (A - Y), \quad (1.20)$$

et le gradient par rapport au biais est :

$$\frac{\partial L}{\partial b} = \frac{1}{m} \sum_{i=1}^m (a_i - y_i). \quad (1.21)$$

La mise à jour vectorisée des paramètres devient :

$$W \leftarrow W - \alpha \frac{\partial L}{\partial W}, \quad b \leftarrow b - \alpha \frac{\partial L}{\partial b}. \quad (1.22)$$

La vectorisation permet donc d'accélérer fortement l'apprentissage, notamment lorsque le nombre de données ou de variables est élevé.

1.3 Un réseau de neurones

1.3.1 Les couches de neurones

Un réseau de neurones est obtenu en combinant plusieurs neurones dans une ou plusieurs couches. Chaque neurone possède ses propres poids et son propre biais. Les sorties produites par une couche peuvent devenir les entrées d'une couche suivante.

Dans une première couche contenant deux neurones, chaque neurone calcule une valeur linéaire z puis une activation a . Pour distinguer les couches, on utilise la notation entre crochets. Ainsi, les grandeurs de la première couche sont notées $Z^{[1]}$ et $A^{[1]}$, tandis que celles de la deuxième couche sont notées $Z^{[2]}$ et $A^{[2]}$.

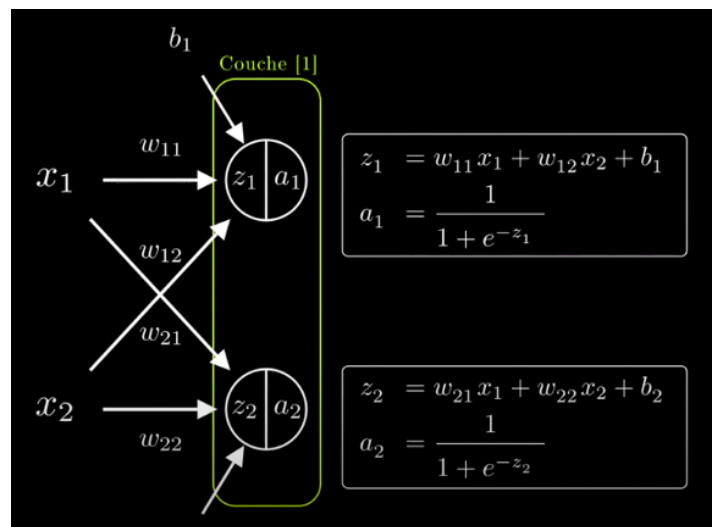
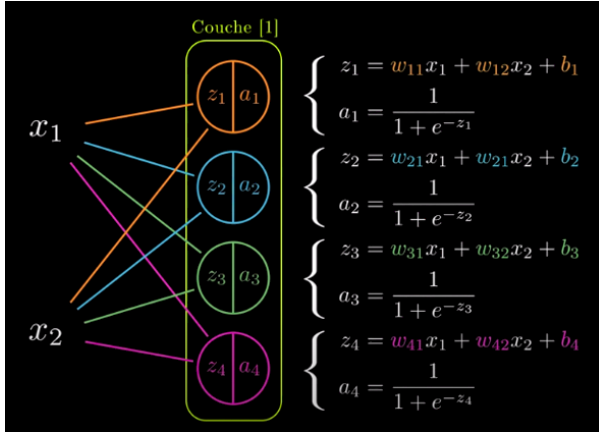
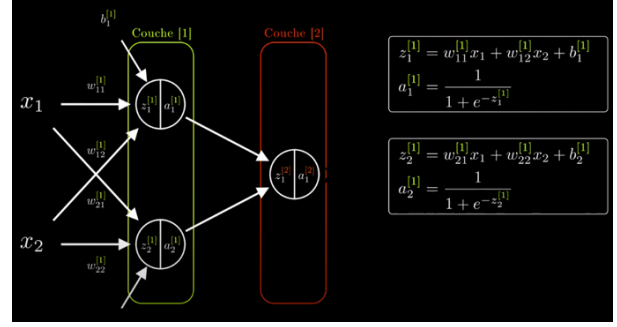


Figure 1.11 – Réseau de neurones à une couche

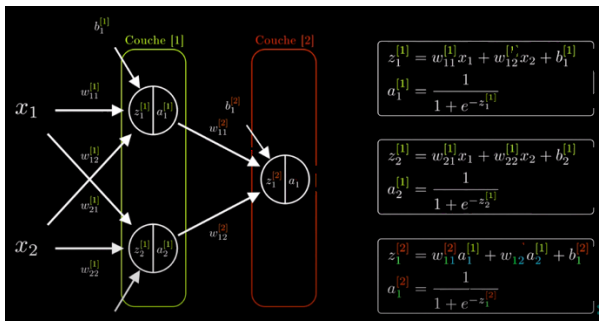


(a) Une couche avec plusieurs neurones.

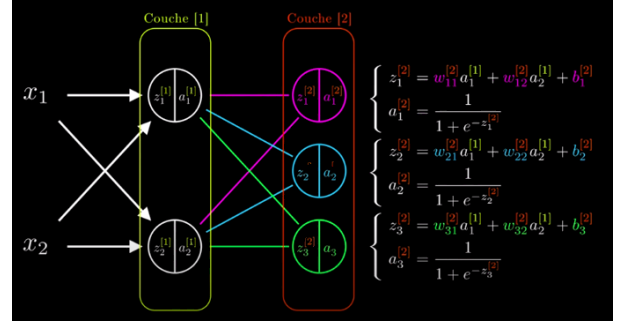


(b) Réseau à deux couches.

Figure 1.12 – Organisation des couches dans un réseau de neurones.



(a) Réseau à deux couches.



(b) Réseau à deux couches et plusieurs neurones.

Figure 1.13 – Ajout progressif de neurones et de couches.

En ajoutant plusieurs couches successives, on obtient un réseau profond, appelé *Deep Neural Network*. Plus le nombre de couches et de neurones augmente, plus le modèle peut représenter des relations complexes. Cependant, cela augmente également le nombre de paramètres et le temps d'apprentissage.

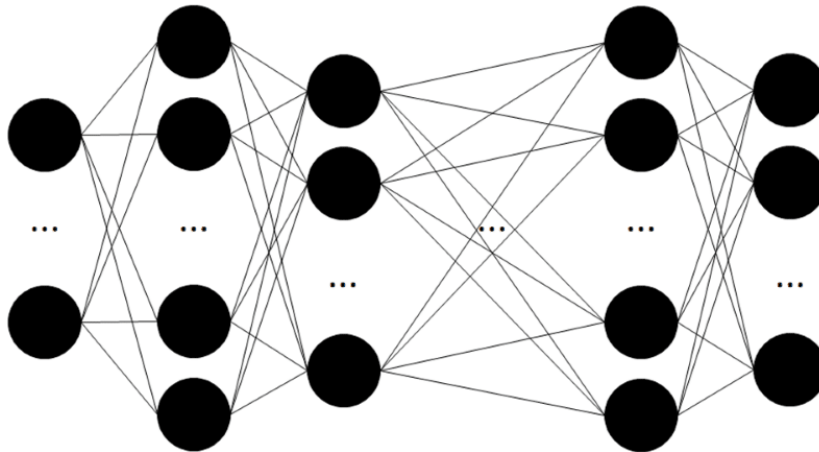


Figure 1.14 – Réseau de neurones à plusieurs couches

1.3.2 La propagation avant

La propagation avant, ou *forward propagation*, correspond au calcul effectué depuis les entrées du réseau jusqu'à la sortie finale. Pour éviter d'écrire séparément les équations de chaque neurone, on représente chaque couche par des matrices.

Considérons une première couche composée de deux neurones recevant deux variables d'entrée. On peut écrire :

$$Z^{[1]} = XW^{[1]} + b^{[1]}, \quad (1.23)$$

où $X \in \mathbb{R}^{m \times 2}$ contient les données, $W^{[1]} \in \mathbb{R}^{2 \times 2}$ contient les poids de la première couche et $b^{[1]} \in \mathbb{R}^{1 \times 2}$ contient les biais.

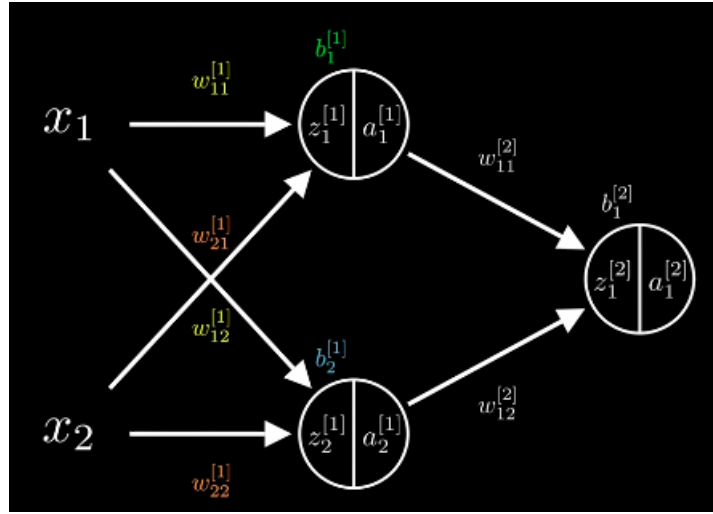


Figure 1.15 – Exemple d'un réseau de neurones

Si l'on ajoute un troisième neurone dans la première couche, la matrice $W^{[1]}$ possède une colonne supplémentaire, la matrice $Z^{[1]}$ comporte également une colonne supplémentaire et le vecteur $b^{[1]}$ contient un biais de plus.

Une autre convention consiste à représenter les observations en colonnes. Dans ce cas, on écrit :

$$Z^{[1]} = W^{[1]}X + b^{[1]}, \quad (1.24)$$

où $W^{[1]} \in \mathbb{R}^{3 \times 2}$, $X \in \mathbb{R}^{2 \times m}$ et $b^{[1]} \in \mathbb{R}^{3 \times 1}$.

Les activations de la première couche sont obtenues en appliquant la fonction d'activation :

$$A^{[1]} = \sigma(Z^{[1]}). \quad (1.25)$$

Ces activations deviennent ensuite les entrées de la deuxième couche :

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}, \quad (1.26)$$

$$A^{[2]} = \sigma(Z^{[2]}). \quad (1.27)$$

De façon générale, pour une couche l , la propagation avant s'écrit :

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}, \quad (1.28)$$

$$A^{[l]} = g^{[l]}(Z^{[l]}), \quad (1.29)$$

où $g^{[l]}$ est la fonction d'activation de la couche l .

1.3.3 La rétropropagation

L'apprentissage d'un réseau de neurones repose sur trois étapes principales. La première consiste à définir une fonction coût mesurant l'erreur entre la sortie prédite et la sortie réelle. La deuxième consiste à calculer les dérivées partielles de cette fonction coût par rapport à tous les paramètres du réseau. La troisième consiste à mettre à jour les paramètres par descente de gradient.

Pour un modèle simple, les mises à jour s'écrivent :

$$W \leftarrow W - \alpha \frac{\partial L}{\partial W}, \quad b \leftarrow b - \alpha \frac{\partial L}{\partial b}. \quad (1.30)$$

Dans le cas d'un réseau comportant plusieurs couches, le principe reste le même. La fonction coût dépend de la sortie finale du réseau. Par exemple, pour une sortie $A^{[2]}$, on peut écrire :

$$L = -\frac{1}{m} \sum \left[Y \log(A^{[2]}) + (1 - Y) \log(1 - A^{[2]}) \right]. \quad (1.31)$$

Il faut ensuite calculer les gradients :

$$\frac{\partial L}{\partial W^{[1]}}, \quad \frac{\partial L}{\partial b^{[1]}}, \quad \frac{\partial L}{\partial W^{[2]}}, \quad \frac{\partial L}{\partial b^{[2]}}. \quad (1.32)$$

Les paramètres sont mis à jour par :

$$W^{[1]} \leftarrow W^{[1]} - \alpha \frac{\partial L}{\partial W^{[1]}}, \quad W^{[2]} \leftarrow W^{[2]} - \alpha \frac{\partial L}{\partial W^{[2]}}, \quad (1.33)$$

$$b^{[1]} \leftarrow b^{[1]} - \alpha \frac{\partial L}{\partial b^{[1]}}, \quad b^{[2]} \leftarrow b^{[2]} - \alpha \frac{\partial L}{\partial b^{[2]}}. \quad (1.34)$$

La rétropropagation consiste à propager l'erreur depuis la sortie du réseau vers les couches précédentes. Pour un réseau à deux couches, les gradients de la deuxième couche peuvent s'écrire sous la forme :

$$\frac{\partial L}{\partial W^{[2]}} = \frac{1}{m} (A^{[2]} - Y) (A^{[1]})^T, \quad (1.35)$$

$$\frac{\partial L}{\partial b^{[2]}} = \frac{1}{m} \sum (A^{[2]} - Y). \quad (1.36)$$

Les gradients de la première couche dépendent de l'erreur propagée depuis la deuxième

couche :

$$\frac{\partial L}{\partial W^{[1]}} = \frac{1}{m} \left[(W^{[2]})^T (A^{[2]} - Y) \odot A^{[1]} (1 - A^{[1]}) \right] X^T, \quad (1.37)$$

$$\frac{\partial L}{\partial b^{[1]}} = \frac{1}{m} \sum \left[(W^{[2]})^T (A^{[2]} - Y) \odot A^{[1]} (1 - A^{[1]}) \right]. \quad (1.38)$$

Le symbole $\odot$ désigne le produit terme à terme. Cette méthode permet d'entraîner efficacement des réseaux comportant plusieurs couches.

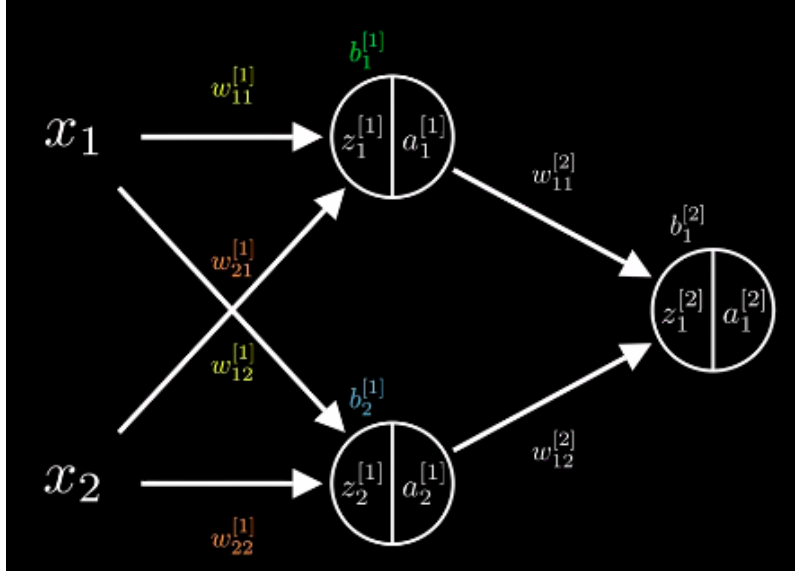
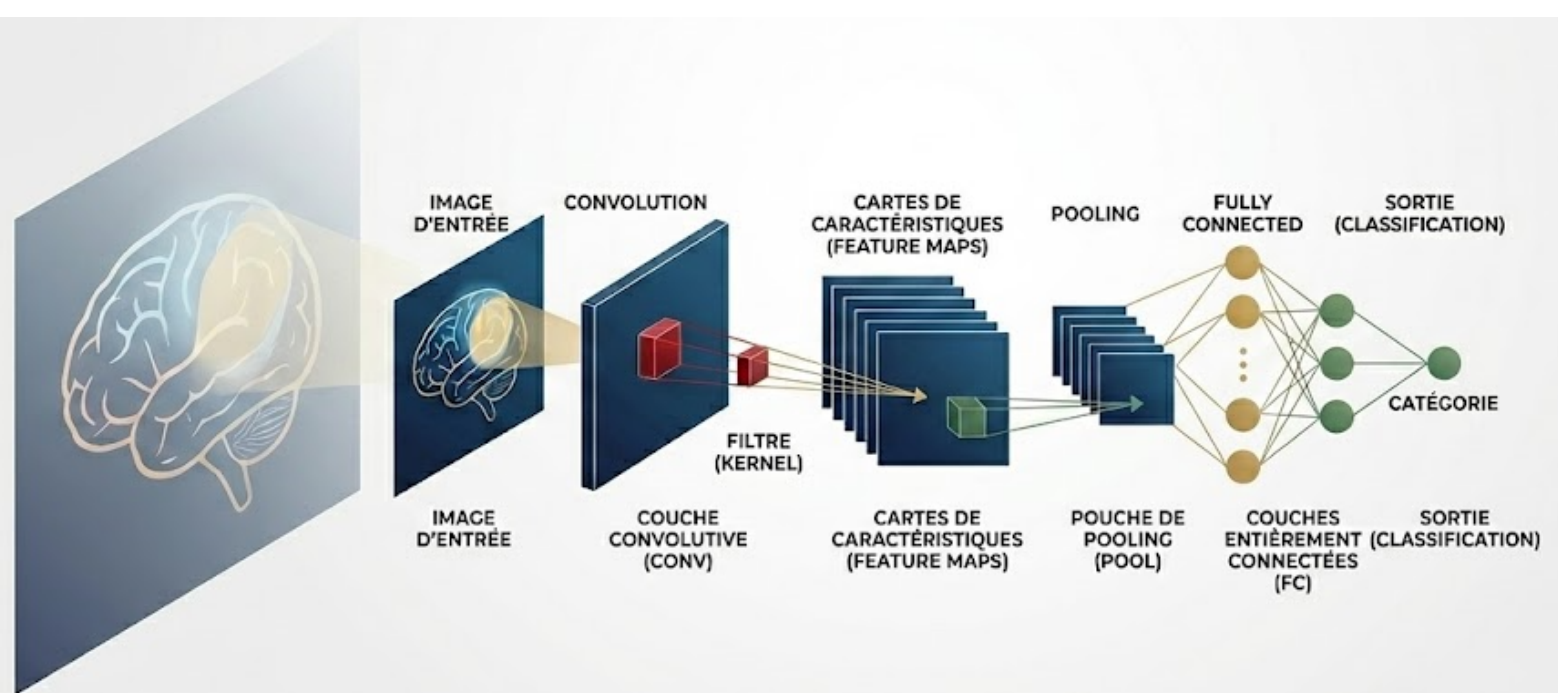


Figure 1.16 – Exemple d'un réseau de neurones

Chapitre 2 :

Réseaux de neurones convolutifs

Convolutional Neural Networks (CNN)



2.1 Introduction

Les réseaux de neurones convolutifs, appelés *Convolutional Neural Networks* ou CNN, sont une famille de réseaux de neurones profonds particulièrement utilisée pour le traitement des images. Leur rôle est d'extraire automatiquement des caractéristiques visuelles, puis d'utiliser ces caractéristiques pour réaliser une tâche comme la classification, la détection d'objets ou la segmentation.

Dans un problème d'analyse d'images, une image couleur est généralement représentée par trois canaux : rouge, vert et bleu. Chaque canal peut être vu comme une matrice de pixels. Une image de taille $n \times m$ possède donc une représentation sous forme de tenseur de dimension $n \times m \times 3$. Cette structure spatiale est importante : deux pixels voisins ont souvent un lien visuel, car ils peuvent appartenir au même contour, à la même texture ou au même objet.

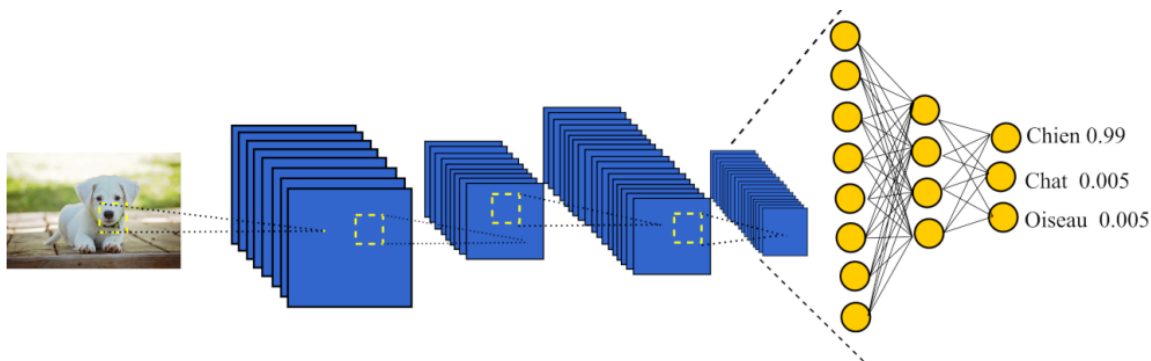


Figure 2.1 – Architecture générale d'un réseau de neurones convolutif pour la classification d'images.

Le but de ce chapitre est de présenter les CNN de manière progressive. On commence par expliquer pourquoi les réseaux entièrement connectés ne sont pas suffisants pour les images. Ensuite, on présente les opérations fondamentales des CNN : la convolution, le padding, le stride, la fonction ReLU, le pooling et les couches entièrement connectées. Enfin, on introduit les notions mathématiques importantes : convolution discrète, corrélation croisée, nombre de paramètres, rétropropagation et champ réceptif.

2.2 Limites des réseaux entièrement connectés pour les images

2.2.1 Rappel sur le perceptron multicouche

Un réseau de neurones classique, ou perceptron multicouche, prend souvent en entrée un vecteur $x \in \mathbb{R}^d$. Pour une couche cachée contenant n neurones, on peut écrire :

$$h = \sigma(Wx + b), \quad (2.1)$$

où W est la matrice des poids, b le vecteur des biais et σ une fonction d'activation non linéaire. La couche de sortie s'écrit ensuite :

$$y = W_y h + b_y. \quad (2.2)$$

Si la sortie est de dimension d' , alors le nombre de paramètres de la couche cachée est :

$$N_{\text{cachée}} = d \times n + n, \quad (2.3)$$

et le nombre de paramètres de la couche de sortie est :

$$N_{\text{sortie}} = n \times d' + d'. \quad (2.4)$$

Ces formules montrent que le nombre de paramètres augmente directement avec la dimension d'entrée d . Or, pour les images, d devient rapidement très grand.

2.2.2 Explosion du nombre de paramètres

Pour une image MNIST de taille 28×28 , la dimension d'entrée est :

$$d = 28 \times 28 = 784. \quad (2.5)$$

Avec 500 neurones cachés et 10 classes, on obtient :

$$28 \times 28 \times 500 + 500 = 392\,500 \quad (2.6)$$

paramètres pour la première couche, puis :

$$500 \times 10 + 10 = 5\,010 \quad (2.7)$$

paramètres pour la couche de sortie. Même sur une petite image en niveaux de gris, le nombre de paramètres est déjà élevé.

Pour une image couleur de webcam de taille $852 \times 480 \times 3$, la dimension d'entrée devient :

$$d = 852 \times 480 \times 3 = 1\,226\,880. \quad (2.8)$$

Avec 500 neurones, le nombre de paramètres est de l'ordre de :

$$1\,226\,880 \times 500 \simeq 614\,000\,000. \quad (2.9)$$

Cela représente plusieurs centaines de millions de poids, donc une mémoire très importante et un apprentissage difficile.

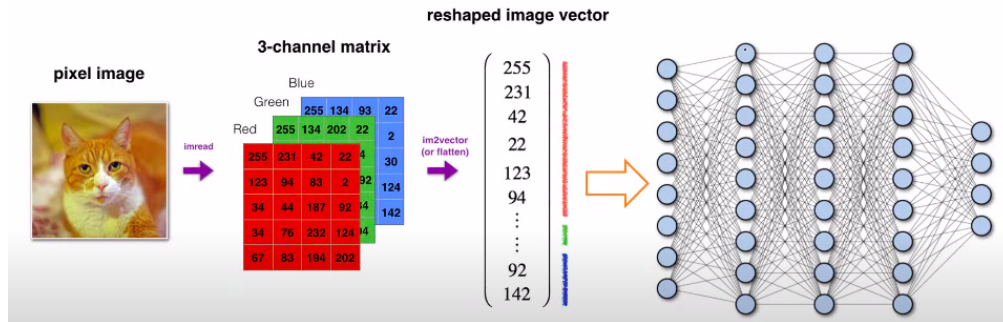


Figure 2.2 – Transformation d’une image en vecteur pour un réseau entièrement connecté.

2.2.3 Perte de la structure spatiale

Lorsqu’une image est aplatie en un vecteur, l’ordre spatial des pixels est moins bien exploité. Un réseau entièrement connecté voit l’image comme une liste de valeurs, alors qu’une image possède une organisation en lignes, colonnes et canaux. Cette organisation est essentielle pour détecter des formes, des contours ou des textures.

Le deuxième problème est que le réseau peut apprendre de mauvaises corrélations. Par exemple, si plusieurs images de chiens ont un fond rouge, un réseau classique peut associer la classe “chien” à la couleur rouge au lieu d’apprendre les caractéristiques propres à l’animal. Les CNN réduisent ce problème en analysant localement l’image par petites régions.

2.3 Intuition et architecture générale d’un CNN

2.3.1 Principe général

Un CNN est conçu pour traiter les images en conservant leur structure spatiale. Contrairement à un réseau entièrement connecté qui connecte toutes les entrées à tous les neurones, un CNN utilise des filtres locaux qui parcourent l’image. Ces filtres détectent des motifs simples dans les premières couches, puis des motifs de plus en plus complexes dans les couches profondes.

Un CNN contient généralement deux grandes parties :

- une partie d’**extraction de caractéristiques**, composée de couches de convolution, de fonctions d’activation et de pooling ;
- une partie de **classification**, composée de couches entièrement connectées après aplatissement des cartes d’activation.

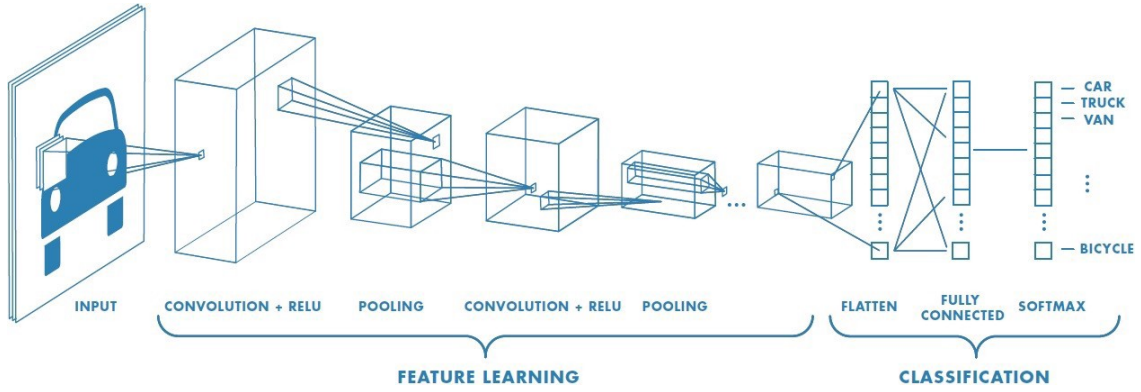
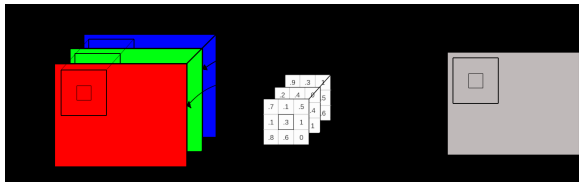


Figure 2.3 – Architecture générale d’un réseau de neurones convolutif : extraction de caractéristiques puis classification.

2.3.2 Rôle des couches principales

La couche de convolution extrait des caractéristiques locales. Par exemple, un filtre peut détecter un bord vertical, un bord horizontal ou une texture. La couche de pooling réduit la taille des cartes obtenues tout en conservant les informations importantes. Les couches entièrement connectées utilisent finalement ces informations pour prédire la classe de l’image.



(a) Couche de convolution.

3	9	7	4	7	5
0	6	7	3	1	2
2	4	5	0	3	2
3	7	5	0	2	1
1	5	0	7	3	6
8	9	2	5	1	8

5	5	4
4	3	2
6	4	5

(b) Couche de pooling.

Figure 2.4 – Les deux opérations de base de la phase d’extraction des caractéristiques.

2.4 A priori sur les images et intérêt des CNN

Les images possèdent une structure locale spatiale. Les pixels voisins ne sont pas indépendants : ils peuvent appartenir au même objet, à la même zone ou au même contour. Les CNN exploitent cette propriété grâce à la connectivité locale.

Un autre a priori important est la stabilité face aux petites transformations. Si un objet se déplace légèrement dans l’image, sa nature ne change pas. Le réseau doit donc produire une représentation similaire. On distingue alors deux notions : l’équivariances et l’invariance.

Une fonction f est **équivariante** par rapport à une transformation g si :

$$f(g(x)) = g(f(x)). \quad (2.10)$$

La convolution est équivariante à la translation : si l’objet est déplacé dans l’image, la carte d’activation est déplacée de la même manière. Une fonction f est **invariante** par

rapport à g si :

$$f(g(x)) = f(x). \quad (2.11)$$

Le pooling introduit une invariance locale, par exemple face à de petites translations.

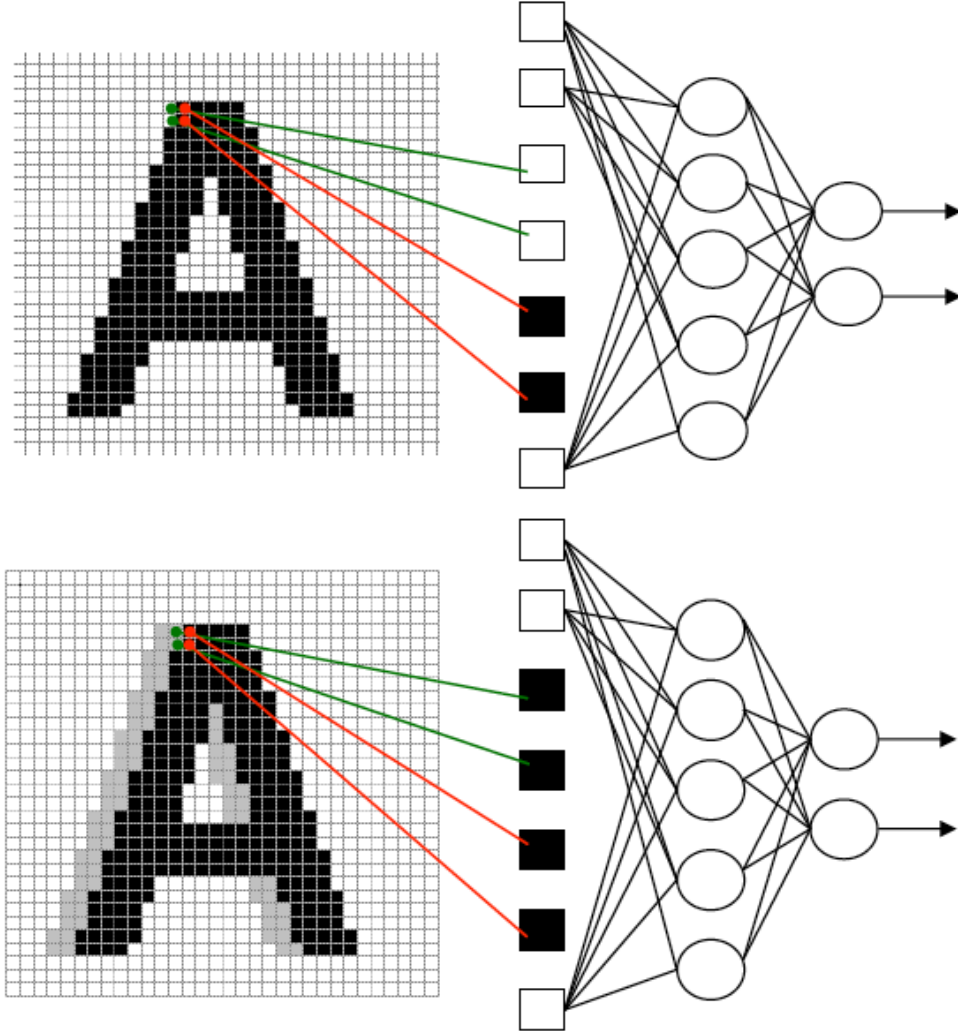


Figure 2.5 – Stabilité des représentations face aux petites déformations et translations.

2.5 Définition mathématique de la convolution

2.5.1 Convolution continue

Soient $f : \mathbb{R} \rightarrow \mathbb{R}$ un signal et $h : \mathbb{R} \rightarrow \mathbb{R}$ un filtre. La convolution entre f et h est définie par :

$$(f * h)(x) = \int_{-\infty}^{+\infty} f(x-t)h(t) dt. \quad (2.12)$$

Cette opération consiste à faire glisser le filtre h sur le signal f et à mesurer localement leur correspondance.

Les propriétés importantes de la convolution sont :

$$f * h = h * f \quad \text{commutativité,} \quad (2.13)$$

$$f * (g + \lambda h) = (f * g) + \lambda(f * h) \quad \text{bilinéarité,} \quad (2.14)$$

$$f * (g * h) = (f * g) * h \quad \text{associativité.} \quad (2.15)$$

2.5.2 Convolution discrète unidimensionnelle

Dans un réseau de neurones, les données sont numériques et donc discrètes. On considère un signal discret f et un filtre h à réponse impulsionnelle finie. Le signal d'entrée est noté :

$$f[i], \quad i \in \{1, \dots, N\}, \quad (2.16)$$

avec N le nombre total d'échantillons du signal. Le filtre est noté :

$$h[n], \quad n \in \left\{ -\frac{d-1}{2}, \dots, \frac{d-1}{2} \right\}, \quad (2.17)$$

avec d la taille du filtre. On choisit souvent d impair afin que le filtre possède un centre. Le signal filtré, ou signal convolué, est noté $f'[i]$. L'opérateur de convolution T_h transforme donc le signal d'entrée f en un nouveau signal :

$$T_h : f \mapsto f' = T_h(f) = f * h. \quad (2.18)$$

La convolution discrète unidimensionnelle est donnée par :

$$T_h(f)[i] = (f * h)[i] = \sum_{n=-\frac{d-1}{2}}^{\frac{d-1}{2}} f[i-n] \cdot h[n]. \quad (2.19)$$

Dans cette formule, i indique la position de sortie où l'on calcule la valeur filtrée, tandis que n indique le décalage local à l'intérieur du filtre. Le terme $f[i-n]$ désigne la valeur du signal située sous le coefficient $h[n]$ du filtre. Chaque valeur voisine est multipliée par un coefficient du filtre, puis toutes les multiplications sont additionnées pour produire la valeur $f'[i]$.

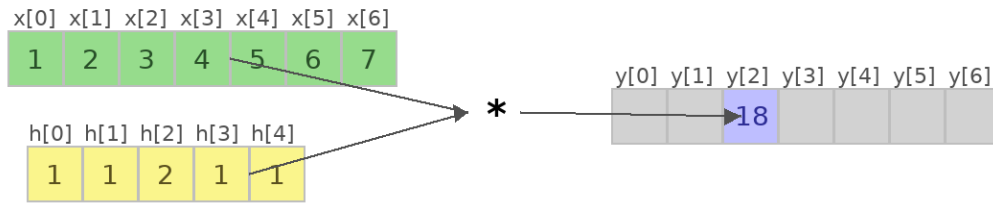


Figure 2.6 – Exemple de convolution discrète 1D : le filtre glisse sur le signal d’entrée et produit une valeur du signal de sortie.

Ainsi, la convolution 1D permet de détecter localement des motifs dans un signal, par exemple une variation rapide dans un son, une tendance dans une série temporelle ou une forme particulière dans une séquence de valeurs.

2.6 Convolution en deux dimensions

2.6.1 Principe général

La convolution en deux dimensions est l’opération fondamentale utilisée dans les CNN pour analyser une image. Contrairement à une couche entièrement connectée, qui utilise tous les pixels en même temps, la convolution observe seulement une petite zone locale de l’image. Cette zone locale est parcourue par un filtre, aussi appelé noyau ou *kernel*.

L’idée est la suivante : pour calculer une nouvelle valeur de sortie au pixel (i, j) , on place le filtre autour de ce pixel, on multiplie chaque valeur du voisinage par le coefficient correspondant du filtre, puis on additionne tous les produits obtenus. Le résultat devient une valeur de la nouvelle image filtrée, appelée aussi **carte d’activation**.

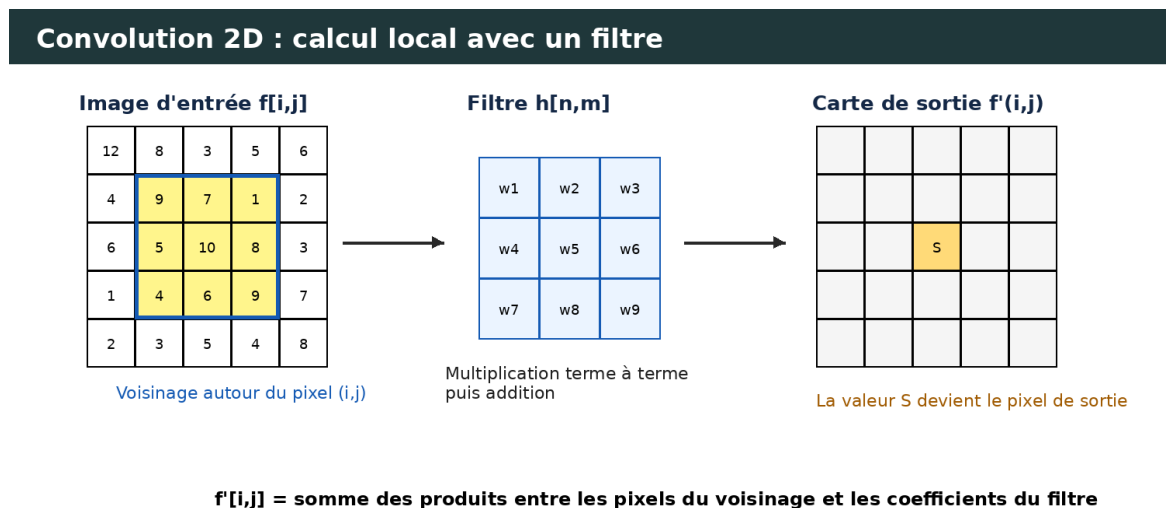


Figure 2.7 – Principe de la convolution 2D : un filtre est appliqué localement sur une région de l’image pour produire une valeur de la carte de sortie.

On considère une image d’entrée f et un filtre h de taille $p \times q$. La convolution discrète 2D s’écrit :

$$f'(i, j) = (f * h)(i, j) = \sum_{n=-\frac{p-1}{2}}^{\frac{p-1}{2}} \sum_{m=-\frac{q-1}{2}}^{\frac{q-1}{2}} f[i - n, j - m] \cdot h[n, m]. \quad (2.20)$$

Les notations utilisées dans cette formule sont importantes :

- $f[i, j]$ désigne la valeur du pixel de l'image d'entrée situé à la ligne i et à la colonne j ;
- $f'(i, j)$ désigne la valeur obtenue dans l'image filtrée, ou carte d'activation, à la position (i, j) ;
- i représente l'indice de ligne du pixel de sortie étudié ;
- j représente l'indice de colonne du pixel de sortie étudié ;
- $h[n, m]$ représente le coefficient du filtre à la position locale (n, m) ;
- p représente la hauteur du filtre, c'est-à-dire son nombre de lignes ;
- q représente la largeur du filtre, c'est-à-dire son nombre de colonnes ;
- n et m sont les indices locaux qui parcourent le filtre. Ils permettent de sélectionner les pixels voisins autour de (i, j) ;
- $f[i - n, j - m]$ est le pixel de l'image qui est multiplié par le coefficient $h[n, m]$.

Ainsi, le filtre ne traite pas l'image entière en une seule fois. Il se déplace progressivement sur toute l'image. À chaque position, il calcule une nouvelle valeur. L'ensemble de ces valeurs forme la carte d'activation. Si le filtre correspond à un motif particulier, comme un contour vertical ou une texture, alors les zones de l'image contenant ce motif produisent des valeurs fortes.

2.6.2 Exemple avec un filtre 3×3

Le cas le plus courant est celui d'un filtre 3×3 . Dans ce cas, le voisinage utilisé autour du pixel (i, j) contient neuf pixels. Si les coefficients du filtre sont notés $w_1, w_2, \dots, w_9$, on obtient :

$$f'[i, j] = w_1 f[i - 1, j - 1] + w_2 f[i - 1, j] + w_3 f[i - 1, j + 1] \quad (2.21)$$

$$+ w_4 f[i, j - 1] + w_5 f[i, j] + w_6 f[i, j + 1] \quad (2.22)$$

$$+ w_7 f[i + 1, j - 1] + w_8 f[i + 1, j] + w_9 f[i + 1, j + 1]. \quad (2.23)$$

Chaque coefficient w_k donne une importance différente au pixel correspondant dans le voisinage. Par exemple, un filtre peut donner plus d'importance aux pixels situés à gauche et à droite pour détecter une variation horizontale, ou aux pixels situés en haut et en bas pour détecter une variation verticale.

Dans un CNN, ces coefficients ne sont pas choisis manuellement. Ils sont appris automa-

tiquement pendant l'entraînement par rétropropagation. Le réseau apprend donc lui-même les filtres qui permettent de reconnaître les motifs utiles pour la tâche considérée.

2.7 Corrélation croisée et interprétation de la convolution

2.7.1 Différence entre convolution et corrélation croisée

En mathématiques, la convolution stricte consiste à retourner le filtre par symétrie centrale avant de l'appliquer sur l'image. Cela signifie que le coefficient situé en haut à gauche du filtre se retrouve en bas à droite, et inversement. Si le filtre initial est noté $h[n, m]$, alors le filtre retourné est :

$$g[n, m] = h[-n, -m]. \quad (2.24)$$

La convolution stricte s'écrit alors sous la forme :

$$f'[i, j] = (f * h)[i, j] = \sum_n \sum_m f[i - n, j - m] h[n, m]. \quad (2.25)$$

La corrélation croisée, elle, applique le filtre directement, sans le retourner :

$$f'[i, j] = (f \star h)[i, j] = \sum_n \sum_m f[i + n, j + m] h[n, m]. \quad (2.26)$$

Convolution ou corrélation croisée ?

Corrélation croisée

Filtre appliqué directement

-4	0	0
0	0	0
0	0	4

$h[n, m]$

retournement
du filtre
→

Convolution stricte

Filtre retourné par symétrie centrale

4	0	0
0	0	0
0	0	-4

$g[n, m] = h[-n, -m]$

Dans les CNN, l'opération appelée « convolution » est souvent une corrélation croisée. Le filtre n'est donc pas obligatoirement retourné, car ses coefficients sont appris.

Figure 2.8 – Différence entre corrélation croisée et convolution stricte : dans la convolution, le filtre est retourné avant application.

Dans les réseaux de neurones convolutifs, l'opération appelée “convolution” correspond très souvent à une corrélation croisée. Cette différence n'est pas problématique, car les

coefficients des filtres sont appris automatiquement. Autrement dit, même si le filtre n'est pas retourné, le réseau peut apprendre les bons coefficients pour détecter les motifs recherchés.

On peut donc retenir l'idée suivante : dans les CNN, la convolution pratique est généralement une opération de corrélation croisée utilisée pour comparer localement une région de l'image avec un filtre appris.

2.7.2 Interprétation comme produit scalaire local

La corrélation croisée peut être comprise comme un **produit scalaire local**. Pour chaque position (i, j) , on extrait une petite région de l'image, par exemple une fenêtre 3×3 , puis on la compare avec un filtre de même taille.

Le calcul consiste à multiplier chaque pixel de la fenêtre par le coefficient correspondant du filtre, puis à additionner tous les produits :

$$(I \star h)[i, j] = \sum_{p, q} I[p, q] \cdot h[p, q]. \quad (2.27)$$

Dans cette écriture, $I[p, q]$ représente les valeurs de la petite région locale de l'image et $h[p, q]$ représente les coefficients du filtre. Le résultat est une seule valeur appelée activation.

Corrélation croisée : produit scalaire local

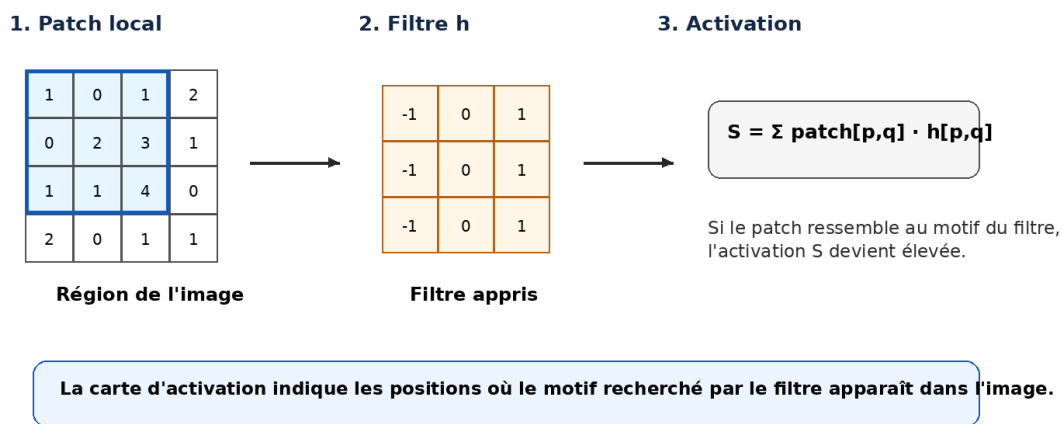


Figure 2.9 – Interprétation de la corrélation croisée comme produit scalaire local entre une région de l'image et un filtre.

Si la région de l'image ressemble au motif représenté par le filtre, le produit scalaire donne une valeur élevée. Si la région ne ressemble pas au motif, la valeur obtenue est faible. C'est pour cette raison que la sortie est appelée carte d'activation : elle indique les positions de l'image où le motif recherché par le filtre est présent.

Par exemple, si un filtre apprend à détecter un bord vertical, alors les régions contenant un bord vertical produiront de fortes activations. Les régions uniformes ou non correspondantes produiront des valeurs faibles. En empilant plusieurs filtres et plusieurs couches convolutives,

le CNN peut apprendre progressivement des motifs simples, comme des contours, puis des motifs plus complexes, comme des parties d'objets ou des formes complètes.

2.8 Padding et stride

2.8.1 Padding

Lorsqu'un filtre est appliqué sur les bords d'une image, une partie du filtre peut sortir de l'image. Pour résoudre ce problème, on peut ajouter un contour artificiel autour de l'image. Cette opération est appelée **padding**. Le cas le plus courant est le **zero-padding**, où les pixels ajoutés valent zéro.

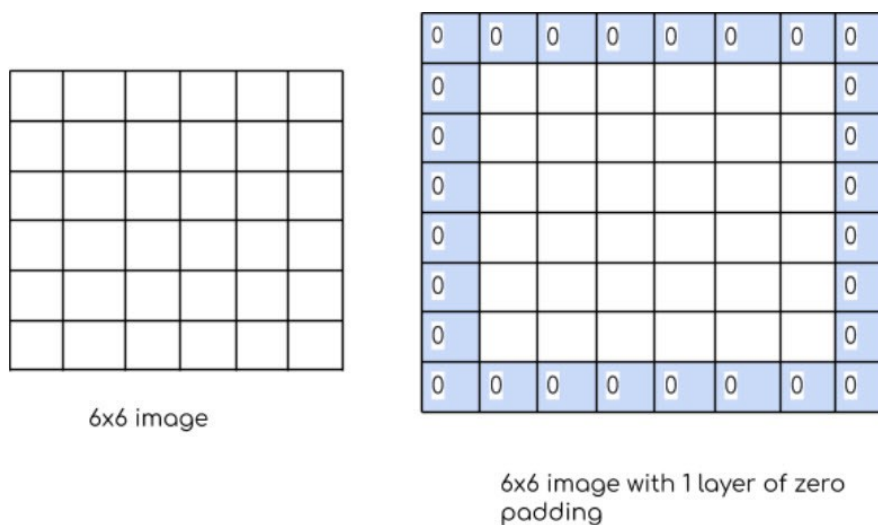


Figure 2.10 – Zero-padding : ajout de pixels nuls autour de l'image pour traiter les bords.

Deux cas sont souvent distingués :

- la convolution **valide**, où l'on ne calcule la sortie que lorsque le filtre est entièrement dans l'image ;
- la convolution avec **padding**, qui permet de conserver la taille de l'image ou de mieux traiter les bords.

Pour une entrée de largeur W , un filtre de taille F , un padding P et un pas S , la taille de sortie est souvent donnée par :

$$W_{\text{out}} = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1. \quad (2.28)$$

De même, pour la hauteur :

$$H_{\text{out}} = \left\lfloor \frac{H - F + 2P}{S} \right\rfloor + 1. \quad (2.29)$$

2.8.2 Stride

Le **stride**, ou pas, indique de combien de pixels le filtre se déplace à chaque étape. Si $s = 1$, le filtre se déplace pixel par pixel. Si $s = 2$, il saute un pixel à chaque déplacement, ce qui réduit la taille de la sortie.

La convolution classique peut s'écrire :

$$T_h(f)[i, j] = \sum_{n=-\frac{d-1}{2}}^{\frac{d-1}{2}} \sum_{m=-\frac{d-1}{2}}^{\frac{d-1}{2}} f[n-i, m-j]h[n, m]. \quad (2.30)$$

Avec un pas s , on obtient :

$$T_h(f)[i, j] = \sum_{n=-\frac{d-1}{2}}^{\frac{d-1}{2}} \sum_{m=-\frac{d-1}{2}}^{\frac{d-1}{2}} f[n-si, m-sj]h[n, m]. \quad (2.31)$$

Le stride agit donc comme un sous-échantillonnage de la carte filtrée.

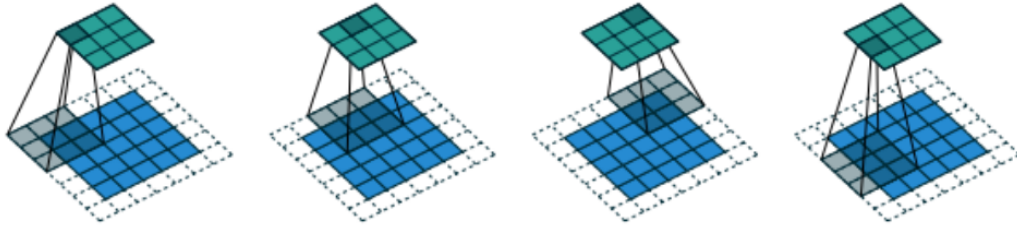


Figure 2.11 – Illustration d'une convolution avec un filtre 3×3 , un padding de 1 et un stride de 2.

2.9 Filtres classiques pour le traitement d'image

Les filtres convolutifs peuvent avoir différentes fonctions. Dans un CNN, ils sont appris automatiquement, mais il est utile de comprendre certains filtres classiques.

2.9.1 Filtre moyennneur

Le filtre moyennneur réalise un lissage de l'image. Il remplace chaque pixel par une moyenne locale :

$$H_{\text{moy}} = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}. \quad (2.32)$$

L'effet principal de ce filtre est d'atténuer les variations brusques entre pixels voisins. L'image obtenue est donc plus douce, mais certains détails fins peuvent être perdus.

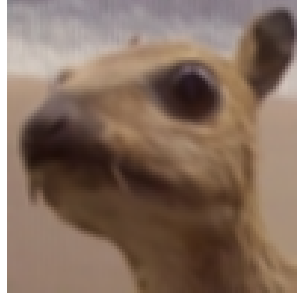


Figure 2.12 – Illustration de l’effet d’un filtre moyennneur : l’image est lissée et les variations locales sont adoucies.

2.9.2 Filtre rehausseur

Le filtre rehausseur accentue les contours :

$$H_{\text{rehausseur}} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}. \quad (2.33)$$

Ce filtre renforce les détails et augmente le contraste local. Il est souvent utilisé pour mettre en évidence les contours et améliorer la netteté visuelle.

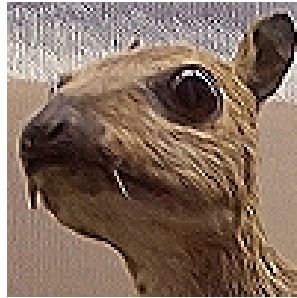


Figure 2.13 – Illustration de l’effet d’un filtre rehausseur : les contours et les détails de l’image sont accentués.

2.9.3 Filtre laplacien

Le filtre laplacien est utilisé pour détecter les contours :

$$H_{\text{laplacien}} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}. \quad (2.34)$$

Le laplacien met en évidence les changements rapides d’intensité. Il produit une image où les contours apparaissent clairement, tandis que les zones uniformes deviennent très sombres.

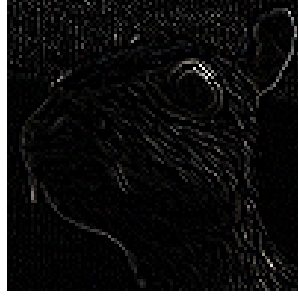


Figure 2.14 – Illustration de l'effet d'un filtre laplacien : les contours sont extraits de l'image.

2.9.4 Filtres de Sobel

Les filtres de Sobel permettent d'approximer les gradients horizontaux et verticaux d'une image. Pour une image I , le gradient est :

$$\vec{G}(x, y) = (G_x, G_y) = \left(\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right). \quad (2.35)$$

La dérivée suivant x peut être approximée par différences finies :

$$\frac{\partial I}{\partial x}[i, j] \approx \frac{1}{2} (I[i + 1, j] - I[i - 1, j]). \quad (2.36)$$

Les matrices de Sobel sont :

$$M_x = \frac{1}{4} \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad M_y = \frac{1}{4} \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}. \quad (2.37)$$

On obtient alors :

$$G_x \approx I \star M_x, \quad G_y \approx I \star M_y. \quad (2.38)$$

La norme du gradient peut être utilisée pour obtenir une image de contours :

$$I_e = \|\vec{G}\| = \sqrt{G_x^2 + G_y^2}. \quad (2.39)$$

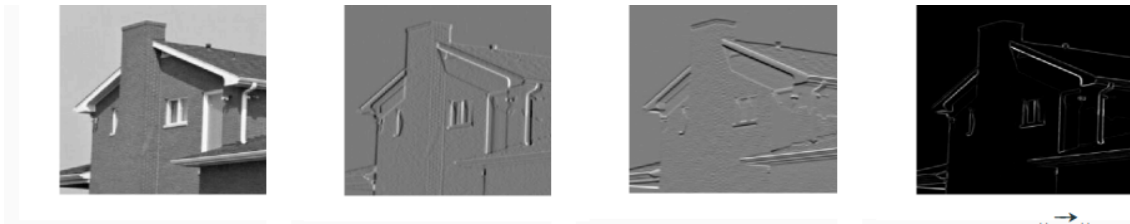


Figure 2.15 – Résultats du filtre de Sobel : de gauche à droite, image source, réponse selon G_x , réponse selon G_y et image finale des contours.

2.10 Fonction d'activation ReLU

Après une convolution, on applique généralement une fonction d'activation non linéaire. Dans les CNN, la fonction la plus utilisée est ReLU, définie par :

$$\text{ReLU}(x) = \max(0, x). \quad (2.40)$$

Elle transforme toutes les valeurs négatives en zéro et conserve les valeurs positives :

$$\text{ReLU}(x) = \begin{cases} 0, & \text{si } x < 0, \\ x, & \text{si } x \geq 0. \end{cases} \quad (2.41)$$

L'intérêt de ReLU est d'introduire une non-linéarité dans le réseau. Sans fonction d'activation, l'empilement de plusieurs couches convolutives resterait équivalent à une seule opération linéaire.

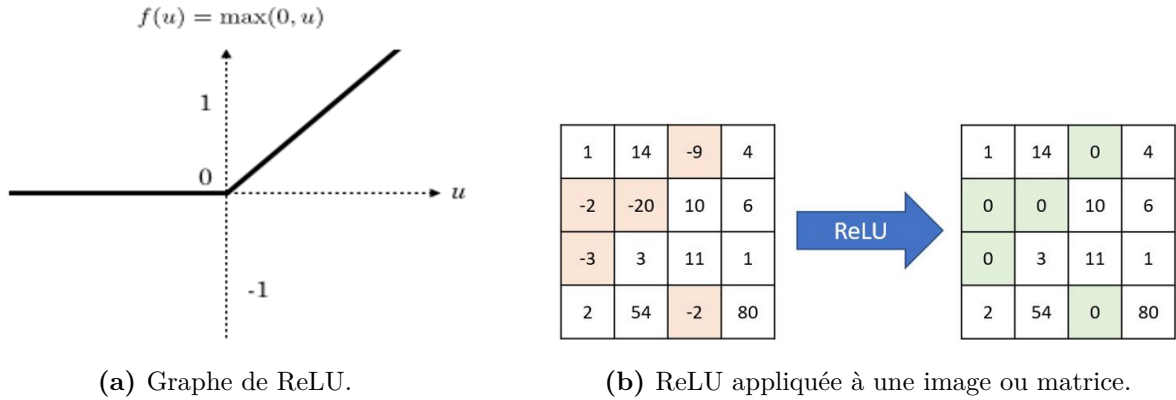


Figure 2.16 – La fonction ReLU conserve les valeurs positives et annule les valeurs négatives.

2.11 Couche convolutive complète

Une couche convolutive est l'élément central d'un CNN. Elle reçoit une image ou un ensemble de cartes d'activation en entrée, applique plusieurs filtres appris, puis produit en sortie de nouvelles cartes d'activation. Chaque carte de sortie met en évidence un type de motif particulier : bord, texture, forme simple ou combinaison de caractéristiques.

2.11.1 Entrée, sortie et paramètres de la couche

L'entrée d'une couche convolutive est un tenseur, c'est-à-dire un tableau à trois dimensions :

$$I \in \mathbb{R}^{C_{in} \times W_{in} \times H_{in}}, \quad (2.42)$$

où :

- C_{in} est le nombre de canaux d'entrée ;
- W_{in} est la largeur de l'entrée ;
- H_{in} est la hauteur de l'entrée.

Par exemple, pour une image couleur RGB, on a généralement $C_{in} = 3$, car l'image possède trois canaux : rouge, vert et bleu. Dans les couches plus profondes du CNN, les canaux ne correspondent plus forcément aux couleurs, mais à des cartes d'activation extraites par les couches précédentes.

La couche convolutive possède ensuite plusieurs filtres. Les paramètres principaux sont :

- C_{out} : le nombre de canaux de sortie, c'est-à-dire le nombre de cartes d'activation produites ;
- k_w : la largeur du filtre ;
- k_h : la hauteur du filtre.

La sortie de la couche est donc un tenseur de dimension :

$$O \in \mathbb{R}^{C_{out} \times W_{out} \times H_{out}}. \quad (2.43)$$

Chaque canal de sortie correspond à une carte d'activation. Si la couche possède $C_{out} = 16$ filtres, alors elle produit 16 cartes d'activation différentes.

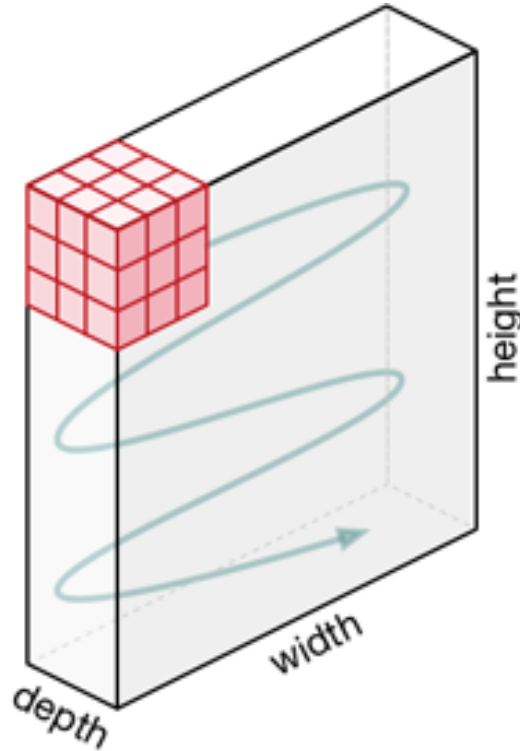


Figure 2.17 – Un filtre convolutif parcourt localement l'image en largeur et en hauteur, tout en prenant en compte la profondeur du tenseur.

2.11.2 Calcul d'une carte d'activation

Pour produire la $j^{\text{ième}}$ carte d'activation de sortie, la couche convolutive applique un filtre sur chaque canal d'entrée, additionne les résultats, puis ajoute un biais. La formule est :

$$o_j = \sum_{i=1}^{C_{in}} I_i \star h_{i,j} + b_j. \quad (2.44)$$

Dans cette expression :

- o_j est la $j^{\text{ième}}$ carte d'activation de sortie ;
- I_i représente le $i^{\text{ième}}$ canal d'entrée ;
- $h_{i,j}$ est le filtre associé au canal d'entrée i et à la carte de sortie j ;
- b_j est le biais associé à la carte de sortie j ;
- $\star$ représente l'opération de corrélation croisée, souvent appelée convolution dans les CNN.

L'idée est donc que chaque carte de sortie ne dépend pas d'un seul canal, mais de l'ensemble des canaux d'entrée. Cela permet au réseau d'apprendre des combinaisons entre les canaux. Par exemple, dans une image RGB, un filtre peut apprendre à détecter une forme à partir d'une combinaison particulière des valeurs rouge, verte et bleue.

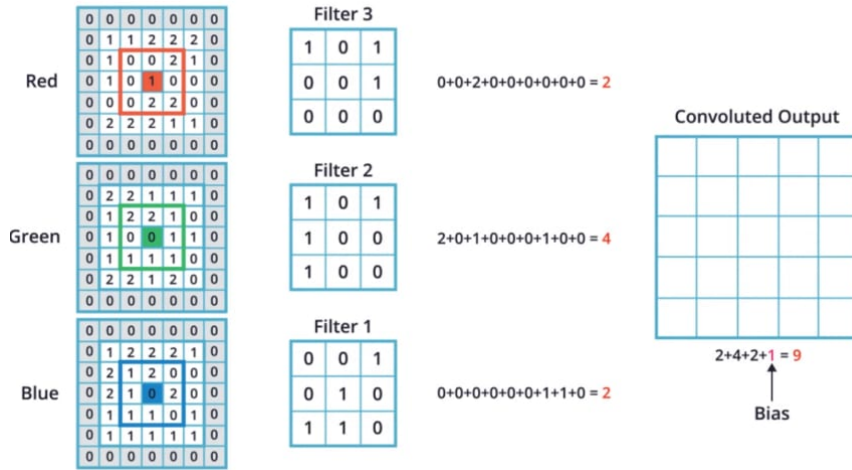


Figure 2.18 – Exemple de convolution sur une image à trois canaux : les réponses obtenues sur chaque canal sont additionnées pour former une carte de sortie.

2.11.3 Nombre de paramètres

Le nombre de paramètres d'une couche convolutive dépend du nombre de filtres, du nombre de canaux d'entrée et de la taille des filtres. Pour une couche ayant C_{out} filtres de taille $k_w \times k_h$ appliqués sur C_{in} canaux d'entrée, on obtient :

$$N_{\text{params}} = C_{\text{out}} \times C_{\text{in}} \times k_w \times k_h + C_{\text{out}}. \quad (2.45)$$

Le terme :

$$C_{\text{out}} \times C_{\text{in}} \times k_w \times k_h \quad (2.46)$$

correspond aux coefficients des filtres, tandis que le terme C_{out} correspond aux biais. Il y a un biais par carte d'activation de sortie.

Cette formule montre l'intérêt des CNN : le nombre de paramètres dépend de la taille des filtres et du nombre de canaux, mais pas directement de la taille complète de l'image. Le même filtre est partagé sur toute l'image, ce qui réduit fortement le nombre de paramètres par rapport à une couche entièrement connectée.

Par exemple, pour une couche avec $C_{\text{in}} = 3$, $C_{\text{out}} = 16$ et des filtres 3×3 , on obtient :

$$N_{\text{params}} = 16 \times 3 \times 3 \times 3 + 16 = 448. \quad (2.47)$$

2.11.4 Convolution sur un tenseur

En pratique, la convolution utilisée dans un CNN est une convolution sur un tenseur. Un filtre ne se limite donc pas à une matrice 2D. Il possède aussi une profondeur égale au nombre de canaux d'entrée. Si l'entrée possède K canaux et si le filtre spatial est de taille $d \times d$, le calcul à la position (i, j) peut s'écrire :

$$T_h[i, j] = (f \star h)[i, j] = \sum_{k=1}^K \sum_{n=-\frac{d-1}{2}}^{\frac{d-1}{2}} \sum_{m=-\frac{d-1}{2}}^{\frac{d-1}{2}} f[i-n, j-m, k] h[n, m, k] + b. \quad (2.48)$$

Les notations sont les suivantes :

- (i, j) indique la position calculée dans la carte de sortie ;
- k parcourt les canaux de l'entrée ;
- n et m parcourent les positions du filtre ;
- $f[i-n, j-m, k]$ est la valeur de l'entrée dans le canal k ;
- $h[n, m, k]$ est le coefficient du filtre correspondant ;
- b est le biais ajouté à la sortie.

Ainsi, pour calculer une seule valeur de sortie, le filtre prend une petite région de l'image, par exemple 3×3 , mais sur tous les canaux d'entrée. Il multiplie chaque valeur par le coefficient correspondant, additionne tous les résultats, puis ajoute le biais.

2.11.5 Exemple numérique d'une couche convolutive

La figure suivante illustre un exemple plus complet. Le volume d'entrée possède une taille $7 \times 7 \times 3$. Cela signifie que l'entrée a une largeur de 7, une hauteur de 7 et une profondeur de 3 canaux. On applique ensuite deux filtres W_0 et W_1 de taille $3 \times 3 \times 3$.

Chaque filtre produit une carte d'activation. Comme il y a deux filtres, on obtient deux cartes de sortie. Le volume de sortie possède donc une profondeur de 2. Dans l'exemple présenté, la sortie est de dimension $3 \times 3 \times 2$.

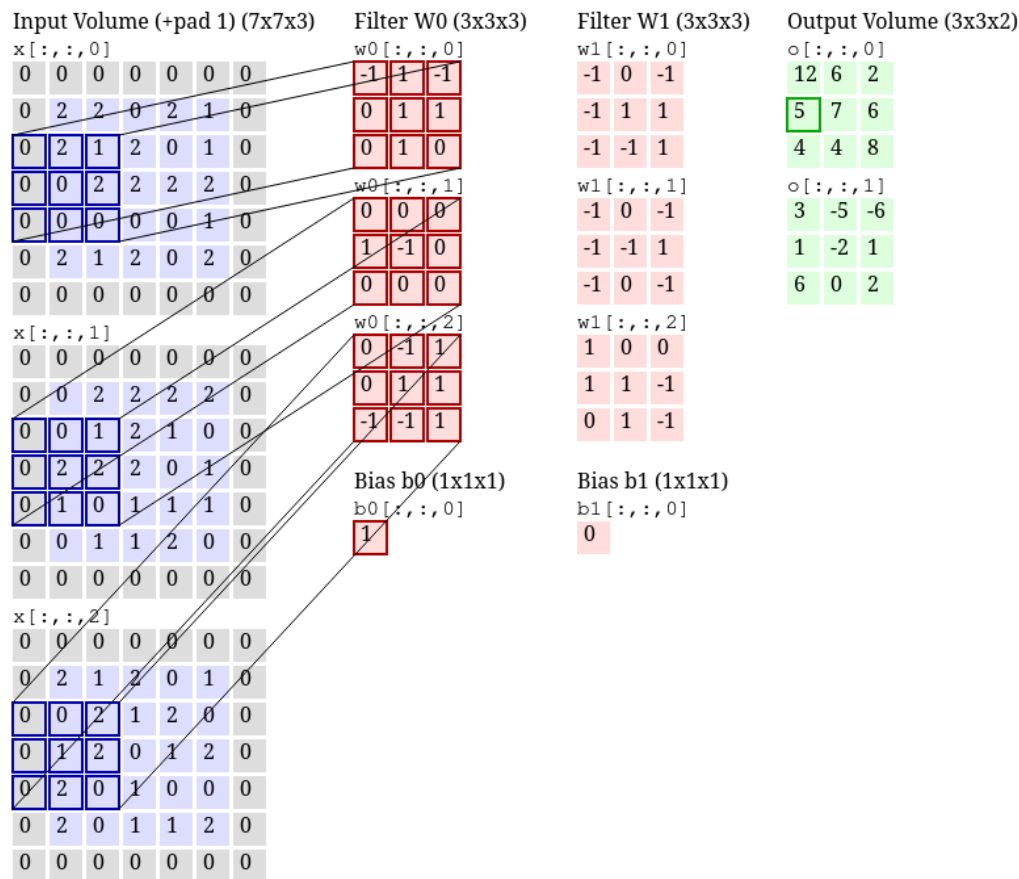


Figure 2.19 – Exemple numérique d'une convolution 3D : deux filtres $3 \times 3 \times 3$ appliqués à un volume d'entrée produisent deux cartes d'activation.

On peut retenir le principe suivant : une région locale de l'entrée est multipliée par un filtre 3D, puis la somme des produits donne une seule valeur dans la carte de sortie. Le filtre se déplace ensuite sur toute l'image afin de calculer les autres valeurs. Plusieurs filtres permettent d'obtenir plusieurs cartes d'activation, chacune spécialisée dans la détection d'un motif particulier.

2.12 Pooling

2.12.1 Définition générale

Le pooling agrège un ensemble de valeurs en une seule valeur représentative. Pour un ensemble :

$$\{x_1, x_2, \dots, x_N\}, \quad (2.49)$$

on produit un scalaire $\tilde{x}$.

Les principales fonctions de pooling sont :

$$\text{Max pooling : } \tilde{x} = \max_{i=1, \dots, N} x_i, \quad (2.50)$$

$$\text{Average pooling : } \tilde{x} = \frac{1}{N} \sum_{i=1}^N x_i, \quad (2.51)$$

$$\ell_p\text{-pooling : } \tilde{x} = \left(\frac{1}{N} \sum_{i=1}^N x_i^p \right)^{1/p}. \quad (2.52)$$

Le pooling réduit la dimension des cartes d'activation et rend le réseau moins sensible à la position exacte d'une caractéristique.

2.12.2 Max pooling et average pooling

Le max pooling conserve la valeur maximale dans une petite région, par exemple un bloc 2×2 . L'average pooling remplace la région par sa moyenne.

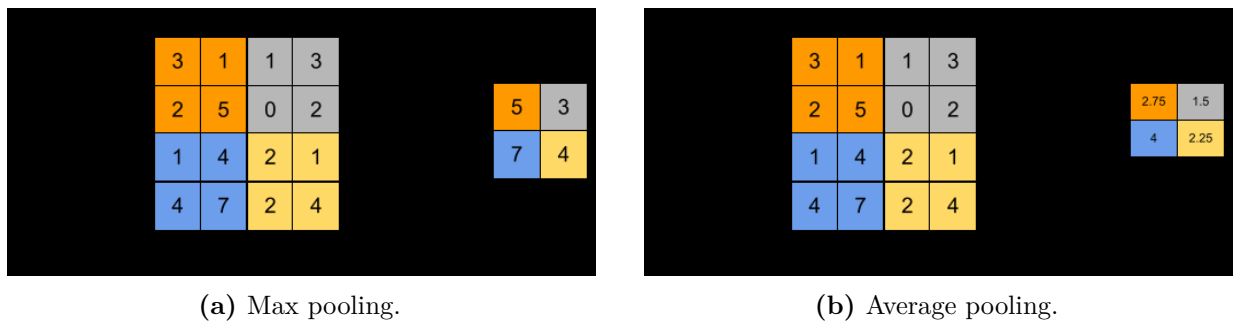


Figure 2.20 – Comparaison entre max pooling et average pooling.

Le pooling spatial est appliqué localement, canal par canal. L'entrée est une carte d'activation, et la sortie est une carte plus petite.

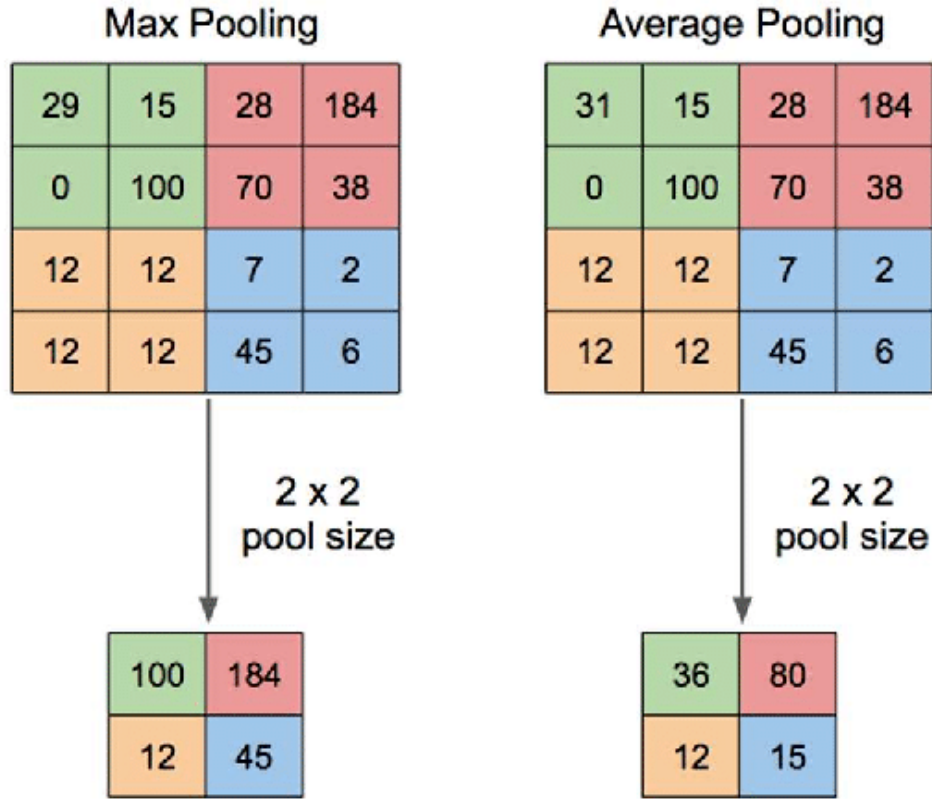


Figure 2.21 – Échantillonnage des cartes filtrées par pooling spatial.

2.13 Équivariance, invariance et rôle du pooling

La convolution est équivariante à la translation : si une caractéristique se déplace dans l'image, son activation se déplace également. Le pooling transforme partiellement cette équivariance en invariance locale. Pour un max pooling, si le maximum reste dans la même fenêtre malgré une petite translation, alors la sortie ne change pas.

Pour une transformation de translation $\vec{T} = (t_x, t_y)$, l'invariance locale est conservée si :

- la translation ne fait pas apparaître un nouveau maximum dans la région ;
- la translation ne fait pas sortir le maximum de la région considérée.

Cette propriété explique pourquoi les CNN sont robustes aux petits déplacements des objets dans les images.

2.14 Rétropropagation dans une couche convolutive

L'apprentissage d'un CNN se fait par rétropropagation du gradient. Pour une convolution simple :

$$s = x \star H, \quad (2.53)$$

où x est l'entrée, H le filtre et s la sortie. Dans cette écriture, H désigne la matrice des poids du filtre convolutif. Pour un filtre de taille $k \times k$, elle peut être représentée sous la forme :

$$H = \begin{bmatrix} h_{11} & h_{12} & \cdots & h_{1k} \\ h_{21} & h_{22} & \cdots & h_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ h_{k1} & h_{k2} & \cdots & h_{kk} \end{bmatrix}. \quad (2.54)$$

Chaque coefficient h_{ab} est un paramètre appris pendant l'entraînement. La rétropropagation permet donc de calculer comment la fonction de coût L varie par rapport à chacun de ces coefficients, afin de mettre à jour le filtre. Les gradients peuvent s'écrire :

$$\frac{\partial L}{\partial H} = X \star \frac{\partial L}{\partial s}, \quad (2.55)$$

et :

$$\frac{\partial L}{\partial x} = H^T \circledast \frac{\partial L}{\partial s}, \quad (2.56)$$

où $\star$ représente la corrélation croisée et $\circledast$ la corrélation croisée complète.

Le point important est que la passe arrière d'une couche convolutive peut elle-même s'interpréter comme une opération convolutive. Cela permet d'entraîner efficacement les filtres du réseau.

2.15 Rétropropagation dans une couche de pooling

Le pooling ne possède pas de paramètres à apprendre. Pendant la rétropropagation, on calcule seulement le gradient par rapport à l'entrée.

2.15.1 Gradient du max pooling

Pour le max pooling :

$$s(i, j) = \max_{p, q} x(p, q). \quad (2.57)$$

Le gradient est transmis uniquement à la position qui a donné la valeur maximale :

$$\frac{\partial L}{\partial x(i, j)} = \begin{cases} \frac{\partial L}{\partial s}, & \text{si } (i, j) = \arg \max_{p, q} x(p, q), \\ 0, & \text{sinon.} \end{cases} \quad (2.58)$$

2.15.2 Gradient de l'average pooling

Pour l'average pooling sur une région $k \times k$:

$$s(i, j) = \frac{1}{k^2} \sum_{p, q} x(p, q). \quad (2.59)$$

Le gradient est réparti entre toutes les positions de la fenêtre :

$$\frac{\partial L}{\partial x(i, j)} = \frac{1}{k^2} \frac{\partial L}{\partial s}. \quad (2.60)$$

2.16 Non-linéarité dans une couche convolutive

Dans les sections précédentes, la fonction ReLU a déjà été présentée comme une fonction d'activation qui conserve les valeurs positives et annule les valeurs négatives. Dans cette partie, l'objectif n'est donc pas de redéfinir ReLU, mais d'expliquer pourquoi une fonction d'activation non linéaire est placée après la convolution dans une couche convolutive.

Une couche convolutive reçoit en entrée un tenseur, par exemple une image ou un ensemble de cartes d'activation. Elle applique d'abord des filtres convolutifs sur des régions locales de ce tenseur. Cette première étape produit des **activations linéaires**, car elle repose principalement sur des multiplications, des additions et l'ajout d'un biais. On peut résumer cette étape par :

$$s = \sum_i x_i w_i + b, \quad (2.61)$$

où x_i représente les valeurs de la région observée, w_i les coefficients du filtre, b le biais et s la valeur obtenue après convolution.

Après cette opération de filtrage, une fonction d'activation non linéaire est appliquée élément par élément sur les valeurs obtenues. Cette étape transforme les activations linéaires en nouvelles cartes d'activation. Elle ne change pas forcément la structure spatiale du tenseur, mais elle modifie les valeurs afin de permettre au réseau d'apprendre des relations plus complexes.

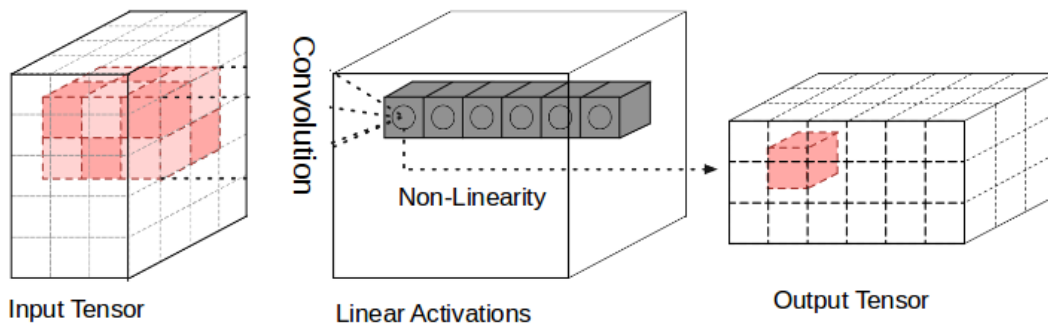


Figure 2.22 – Une couche convolutive transforme un tenseur d'entrée en tenseur de sortie grâce à deux étapes : une convolution affine suivie d'une activation non linéaire.

L'intérêt principal de cette non-linéarité est d'augmenter la capacité d'apprentissage du réseau. Si plusieurs couches convolutives étaient empilées sans fonction d'activation, l'ensemble du réseau resterait équivalent à une seule transformation linéaire. Dans ce cas, même avec beaucoup de couches, le modèle aurait une capacité limitée pour représenter des formes complexes.

Grâce à la non-linéarité, chaque couche peut extraire des informations plus riches que la précédente. Les premières couches peuvent détecter des motifs simples comme des bords ou des textures, tandis que les couches plus profondes peuvent combiner ces motifs pour construire des représentations plus abstraites. C'est cette combinaison entre convolution et activation non linéaire qui permet aux CNN d'apprendre progressivement des caractéristiques utiles pour la classification ou la détection d'objets.

On peut donc résumer une couche convolutive complète par la chaîne suivante :

$$\text{tenseur d'entrée} \longrightarrow \text{convolution affine} \longrightarrow \text{activation non linéaire} \longrightarrow \text{tenseur de sortie.} \quad (2.62)$$

2.17 Hiérarchie de convolutions et champ réceptif

L'une des idées les plus importantes dans les CNN est l'**empilement des couches convolutives**. Un réseau convolutif ne se limite pas à appliquer un seul filtre sur l'image. Il applique plusieurs couches successives, où chaque couche reçoit les cartes d'activation produites par la couche précédente. Ainsi, le réseau construit progressivement une représentation de plus en plus riche de l'image.

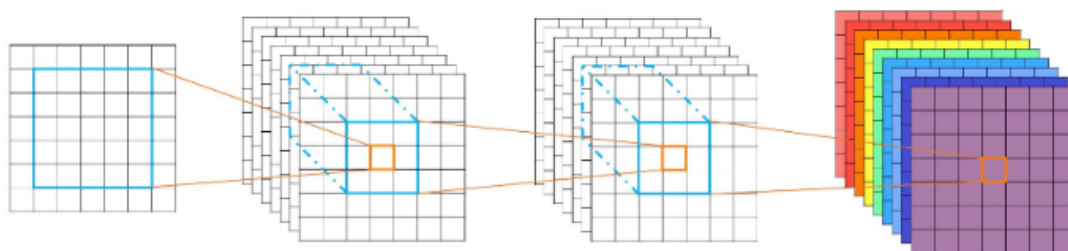


Figure 2.23 – Principe de la hiérarchie de convolutions : les couches successives combinent progressivement des informations locales pour construire des représentations plus complexes.

2.17.1 Principe de la hiérarchie de convolutions

Dans les premières couches, les filtres détectent souvent des motifs très simples : bords, lignes, coins, petites variations d'intensité ou textures élémentaires. Ces informations sont locales, car chaque filtre observe seulement une petite région de l'image.

Dans les couches suivantes, le réseau combine ces motifs simples pour former des motifs plus complexes. Par exemple, plusieurs bords peuvent être combinés pour former une forme, puis plusieurs formes peuvent être combinées pour représenter une partie d'objet. Dans

les couches profondes, le réseau peut donc apprendre des représentations beaucoup plus abstraites.

On peut résumer cette progression ainsi :

$$\text{pixels} \rightarrow \text{bords} \rightarrow \text{formes simples} \rightarrow \text{parties d'objets} \rightarrow \text{objets}. \quad (2.63)$$

Cette progression explique pourquoi les CNN sont efficaces en vision par ordinateur : ils ne cherchent pas directement à reconnaître un objet à partir des pixels bruts, mais construisent progressivement des caractéristiques intermédiaires.

2.17.2 Exemple : détection de contours par deux couches

Pour illustrer cette hiérarchie, on peut considérer une image en niveaux de gris notée :

$$I \in \mathbb{R}^{W \times H \times 1}, \quad (2.64)$$

où W représente la largeur, H la hauteur et 1 indique qu'il n'y a qu'un seul canal, car l'image est en niveaux de gris.

Une première couche peut appliquer deux filtres de Sobel : un filtre M_x pour détecter les variations selon l'axe horizontal et un filtre M_y pour détecter les variations selon l'axe vertical :

$$M_x = \frac{1}{4} \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad M_y = \frac{1}{4} \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}. \quad (2.65)$$

La première couche produit alors deux cartes d'activation :

$$G_x^2 \approx I \star M_x, \quad G_y^2 \approx I \star M_y. \quad (2.66)$$

La carte G_x^2 met en évidence certains contours de l'image, tandis que la carte G_y^2 met en évidence d'autres contours. Ces deux cartes contiennent donc des informations complémentaires.



Figure 2.24 – Première couche de l'exemple : application de deux filtres de Sobel sur une image en niveaux de gris pour obtenir deux cartes de contours.

Une deuxième couche peut ensuite combiner ces deux cartes avec un filtre 1×1 de poids $[+1, +1]$. Ce filtre additionne les informations extraites par les deux filtres précédents :

$$\text{Sortie} = G_x^2 + G_y^2. \quad (2.67)$$

La sortie finale contient alors une représentation plus complète des contours de l'image. Cet exemple montre bien le rôle de l'empilement : la première couche extrait des informations simples, puis la deuxième couche les combine pour produire une information plus globale.

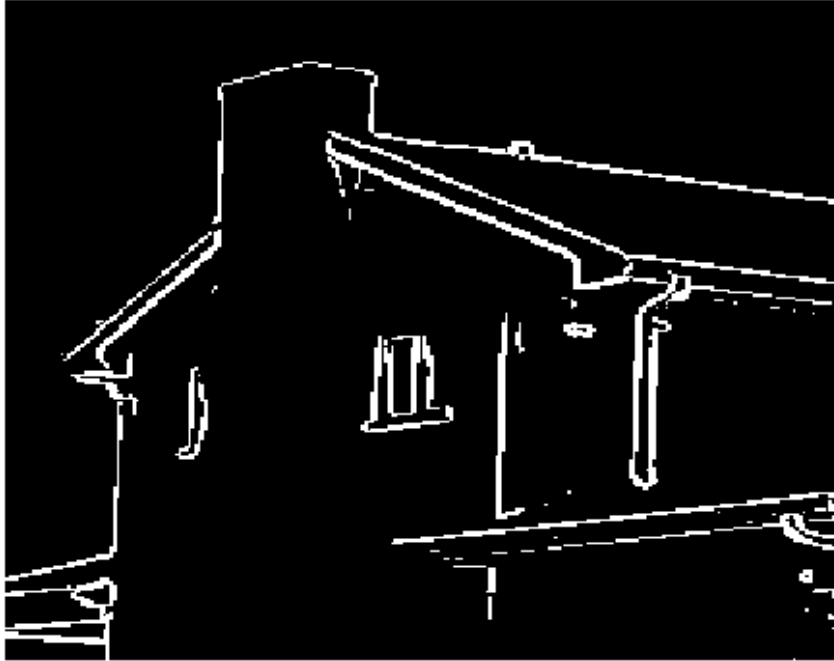


Figure 2.25 – Deuxième couche de l'exemple : combinaison des deux cartes de contours pour produire une sortie plus complète.

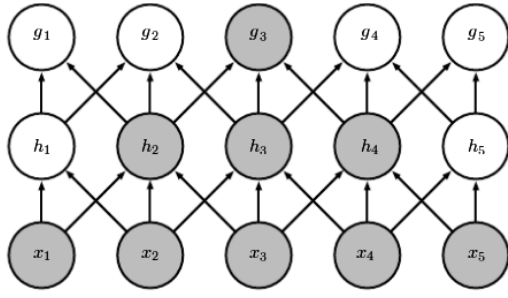
2.17.3 Champ réceptif et échantillonnage des cartes d'activation

Dans un réseau convolutif, deux mécanismes jouent un rôle important dans la construction progressive des représentations : le **champ réceptif** et l'**échantillonnage** des cartes d'activation. Le premier explique quelle zone de l'image influence une activation donnée, tandis que le second explique comment le pooling réduit la taille spatiale des cartes obtenues après convolution.

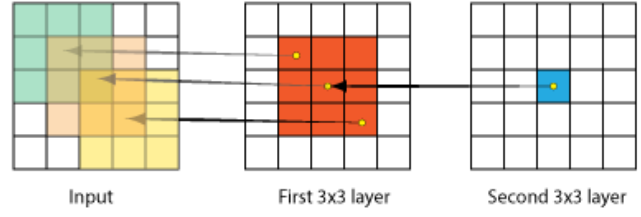
Champ réceptif

Le **champ réceptif** correspond à la région de l'image d'entrée qui contribue au calcul d'une activation dans une couche donnée. Autrement dit, lorsqu'un neurone d'une couche profonde produit une valeur, cette valeur dépend d'une certaine zone de l'image de départ.

Dans une première couche convolutive, le champ réceptif est généralement local. Par exemple, avec un filtre de taille 3×3 , chaque activation dépend directement d'un voisinage de 3×3 pixels. Cependant, lorsque plusieurs couches convolutives sont empilées, le champ réceptif augmente progressivement. Ainsi, une activation d'une deuxième couche dépend indirectement d'une région plus grande de l'image initiale.



(a) Propagation de l'information entre plusieurs couches.



(b) Deux convolutions 3×3 augmentent le champ réceptif.

Figure 2.26 – Illustration du champ réceptif : une activation profonde dépend indirectement d'une zone plus large de l'image d'entrée.

Un point important est que l'empilement de deux convolutions 3×3 permet d'obtenir un champ réceptif équivalent à une convolution 5×5 sur l'image d'origine. Cette stratégie est très utilisée dans les CNN, car elle permet d'augmenter la zone observée tout en conservant des filtres de petite taille.

On peut retenir l'idée suivante :

$$\text{plus le réseau est profond, plus le champ réceptif augmente.} \quad (2.68)$$

Ainsi, les premières couches observent de petites zones et détectent des motifs simples comme des bords ou des textures. Les couches plus profondes combinent ensuite ces informations pour analyser des formes plus grandes et plus complexes. Même si chaque couche travaille localement, l'ensemble du réseau obtient progressivement une vision plus globale de l'image.

Échantillonnage par pooling

Après une ou plusieurs couches convolutives, on obtient des cartes d'activation. Ces cartes peuvent contenir beaucoup de valeurs, surtout lorsque l'image d'entrée est grande. Le **pooling** permet alors de réduire leur taille spatiale tout en conservant les informations les plus importantes.

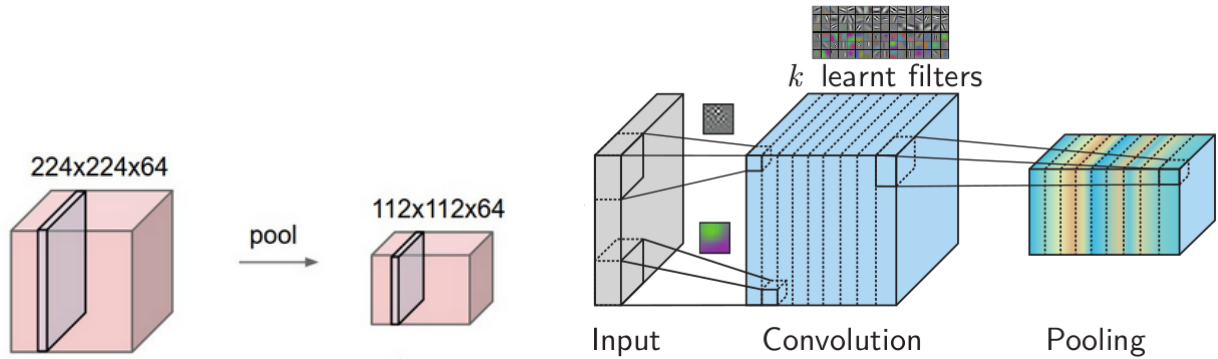
Le pooling est généralement appliqué canal par canal. Cela signifie qu'il ne mélange pas les différents canaux entre eux : il réduit uniquement la largeur et la hauteur de chaque carte d'activation. Par exemple, une carte de taille $224 \times 224 \times 64$ peut devenir, après pooling, une carte de taille :

$$112 \times 112 \times 64. \quad (2.69)$$

Dans cet exemple, la largeur et la hauteur sont divisées par 2, tandis que le nombre de canaux reste égal à 64. De manière générale, si le tenseur d'entrée est de taille (C, W, H) et que le pooling utilise un pas k , alors la sortie peut être écrite approximativement sous la

forme :

$$\left(C, \frac{W}{k}, \frac{H}{k}\right). \quad (2.70)$$



(a) Réduction de la taille spatiale par pooling.

(b) Pooling appliqué après une couche de convolution.

Figure 2.27 – Échantillonnage des cartes d’activation : le pooling réduit la largeur et la hauteur, tout en conservant le nombre de canaux.

L’intérêt du pooling est multiple. Il réduit le nombre de calculs, diminue la quantité de mémoire nécessaire et rend le réseau moins sensible aux petits déplacements des motifs dans l’image. Par exemple, si un contour ou une texture se déplace légèrement dans une région locale, le résultat du pooling peut rester très proche.

On peut donc résumer cette partie ainsi : le champ réceptif permet au réseau d’analyser des zones de plus en plus grandes lorsque la profondeur augmente, tandis que le pooling réduit progressivement la taille des cartes d’activation pour rendre le réseau plus efficace et plus robuste.

2.18 Couches entièrement connectées et classification

Après plusieurs blocs convolution-pooling, les cartes d’activation contiennent les caractéristiques importantes de l’image. Pour réaliser une classification, ces cartes sont généralement aplaties en un vecteur. Cette opération est appelée **flattening**.

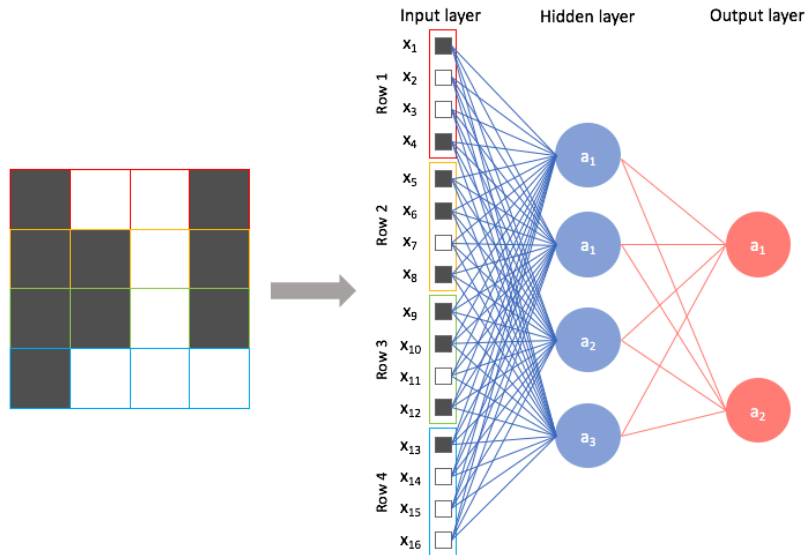


Figure 2.28 – Flattening : transformation des cartes d’activation en vecteur avant les couches entièrement connectées.

Les couches entièrement connectées utilisent ensuite ce vecteur pour produire une probabilité par classe. Par exemple, pour distinguer deux classes, chat et chien, la sortie peut être :

$$[0.9, 0.1], \quad (2.71)$$

ce qui signifie que le modèle estime une probabilité de 90% pour la classe chat et 10% pour la classe chien.

2.19 Exemple d’architecture : LeNet-5

LeNet-5 est une architecture historique proposée par LeCun et al. en 1998 pour la reconnaissance automatique de chiffres manuscrits. Même si elle est plus simple que les architectures modernes, elle reste très importante, car elle montre clairement l’organisation générale d’un réseau de neurones convolutif : une première partie extrait les caractéristiques visuelles de l’image, puis une seconde partie réalise la classification.

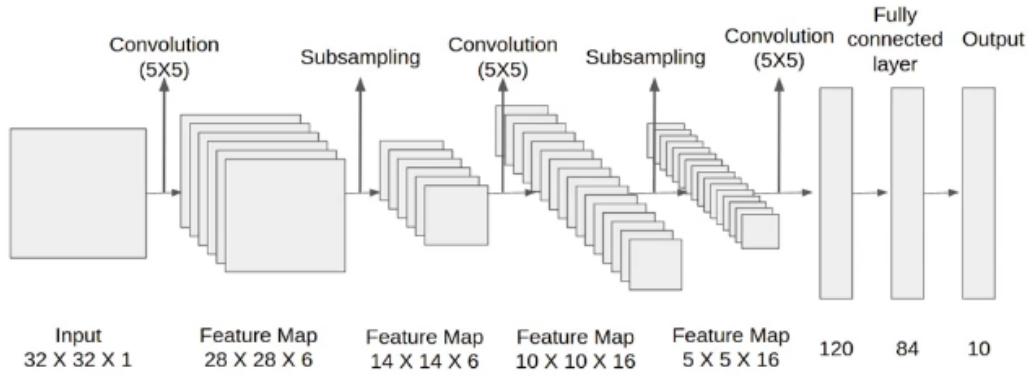


Figure 2.29 – Architecture générale de LeNet-5 : succession de convolutions, de sous-échantillonnages et de couches entièrement connectées.

2.19.1 Principe général de l’architecture

LeNet-5 reçoit en entrée une image en niveaux de gris de dimension :

$$32 \times 32 \times 1. \quad (2.72)$$

Le troisième terme vaut 1 parce que l’image possède un seul canal. Dans le cas des chiffres manuscrits, l’objectif est de classer l’image dans l’une des dix classes possibles :

$$0, 1, 2, 3, 4, 5, 6, 7, 8, 9. \quad (2.73)$$

L’architecture peut être décomposée en deux blocs principaux. Le premier bloc correspond à l’extraction des caractéristiques. Il contient des couches de convolution et de sous-échantillonnage. Le second bloc correspond à la classification. Il contient des couches entièrement connectées qui utilisent les caractéristiques extraites pour produire la sortie finale.

2.19.2 Première convolution et premier sous-échantillonnage

La première couche de convolution utilise des filtres de taille 5×5 . Elle transforme l’image d’entrée selon :

$$32 \times 32 \times 1 \rightarrow 28 \times 28 \times 6. \quad (2.74)$$

La taille spatiale passe de 32×32 à 28×28 , car une convolution 5×5 sans padding réduit la largeur et la hauteur selon :

$$32 - 5 + 1 = 28. \quad (2.75)$$

Le nombre de canaux de sortie vaut 6, car cette couche utilise six filtres différents.

Chaque filtre apprend à détecter un motif simple, par exemple un bord, une courbe ou une petite variation locale d'intensité.

Après cette convolution, une couche de sous-échantillonnage réduit la taille des cartes d'activation :

$$28 \times 28 \times 6 \rightarrow 14 \times 14 \times 6. \quad (2.76)$$

Cette opération réduit le nombre de calculs et rend le réseau moins sensible aux petits déplacements des motifs dans l'image. Le nombre de canaux reste égal à 6, car le sous-échantillonnage est appliqué séparément sur chaque carte d'activation.

2.19.3 Deuxième convolution et deuxième sous-échantillonnage

La deuxième couche de convolution utilise aussi des filtres de taille 5×5 . Elle transforme les cartes précédentes selon :

$$14 \times 14 \times 6 \rightarrow 10 \times 10 \times 16. \quad (2.77)$$

La taille spatiale passe de 14×14 à 10×10 , car :

$$14 - 5 + 1 = 10. \quad (2.78)$$

Le nombre de cartes passe de 6 à 16. Cela signifie que cette couche apprend seize filtres capables de combiner les motifs simples détectés dans la première couche. À ce niveau, le réseau commence à représenter des formes plus complexes, comme des arcs, des jonctions ou des parties de chiffres.

Ensuite, une deuxième opération de sous-échantillonnage réduit à nouveau la dimension spatiale :

$$10 \times 10 \times 16 \rightarrow 5 \times 5 \times 16. \quad (2.79)$$

Le réseau conserve donc 16 cartes de caractéristiques, mais chacune devient plus petite. Cela permet de garder l'information essentielle tout en diminuant la complexité du modèle.

2.19.4 Troisième convolution et passage vers la classification

La troisième couche de convolution utilise des filtres de taille 5×5 appliqués sur les cartes de taille $5 \times 5 \times 16$. Elle produit :

$$5 \times 5 \times 16 \rightarrow 1 \times 1 \times 120. \quad (2.80)$$

Cette étape peut être vue comme une transition entre la partie convolutionnelle et la

partie entièrement connectée. Les filtres couvrent toute la surface spatiale restante, ce qui permet de résumer les caractéristiques extraites en un vecteur de 120 valeurs.

Ce vecteur est ensuite transmis à une couche entièrement connectée :

$$120 \rightarrow 84. \quad (2.81)$$

Cette couche combine les informations extraites par les convolutions afin de préparer la décision finale. Enfin, la couche de sortie transforme les 84 valeurs en 10 scores :

$$84 \rightarrow 10. \quad (2.82)$$

Les dix sorties correspondent aux dix classes possibles. La classe associée au score le plus élevé est choisie comme prédiction du réseau.

2.19.5 Résumé de LeNet-5

On peut résumer l'architecture complète par la chaîne suivante :

$$\begin{aligned} 32 \times 32 \times 1 &\rightarrow 28 \times 28 \times 6 \\ &\rightarrow 14 \times 14 \times 6 \\ &\rightarrow 10 \times 10 \times 16 \\ &\rightarrow 5 \times 5 \times 16 \\ &\rightarrow 1 \times 1 \times 120 \\ &\rightarrow 84 \rightarrow 10. \end{aligned} \quad (2.83)$$

Cet exemple montre bien la logique d'un CNN : les premières couches extraient des motifs simples, les couches suivantes combinent ces motifs pour obtenir des représentations plus riches, puis les couches entièrement connectées réalisent la classification. LeNet-5 constitue donc une base importante pour comprendre les architectures convolutives modernes utilisées aujourd'hui en vision par ordinateur.

2.20 Synthèse du fonctionnement d'un CNN

Un réseau de neurones convolutif peut être résumé par la chaîne suivante :

$$\text{Image} \rightarrow \text{Convolutions} \rightarrow \text{ReLU} \rightarrow \text{Pooling} \rightarrow \text{Flattening} \rightarrow \text{Fully connected} \rightarrow \text{Classe}. \quad (2.84)$$

La convolution extrait les motifs locaux. ReLU introduit la non-linéarité. Le pooling réduit la dimension et apporte une invariance locale. Le flattening transforme les cartes en vecteur. Les couches fully connected réalisent la décision finale.

2.21 Conclusion

Les réseaux de neurones convolutifs constituent une architecture fondamentale du deep learning pour le traitement d'images. Ils ont été développés pour répondre aux limites des réseaux entièrement connectés : nombre trop élevé de paramètres, perte de la structure spatiale et mauvaise prise en compte des motifs locaux.

Grâce aux filtres convolutifs, un CNN apprend automatiquement les caractéristiques utiles d'une image. Les premières couches détectent des motifs simples comme les bords ou les textures, puis les couches profondes combinent ces informations pour représenter des objets plus complexes. Les opérations de padding, de stride et de pooling permettent de contrôler la taille des cartes d'activation et de renforcer la robustesse du modèle.

Du point de vue mathématique, les CNN reposent sur des opérations bien définies : convolution discrète, corrélation croisée, produit scalaire local, fonctions d'activation, pooling et rétropropagation. Ces opérations permettent d'entraîner les filtres par descente de gradient comme dans les réseaux de neurones classiques.

Chapitre 3

Application d'un CNN sur la base MNIST

3.1 Introduction

Après avoir étudié le fonctionnement des réseaux de neurones convolutifs, nous avons réalisé une application pratique afin de mieux comprendre leur utilisation dans un problème de classification d'images. Pour cette application, nous avons choisi la base MNIST, composée d'images de chiffres manuscrits allant de 0 à 9.

L'objectif est de construire un modèle CNN capable de recevoir une image en entrée et de prédire automatiquement le chiffre correspondant. Nous avons organisé le travail en plusieurs étapes : chargement des données, prétraitement, création du modèle, entraînement, évaluation et analyse des résultats.

3.2 Environnement de travail

Nous avons réalisé cette application avec Google Colab, car cet environnement permet d'exécuter un notebook Python directement en ligne. Les bibliothèques principales utilisées sont TensorFlow/Keras pour la création du modèle, NumPy pour la manipulation des données, Matplotlib et Seaborn pour l'affichage des résultats, ainsi que Scikit-learn pour l'évaluation.

Outil	Utilisation
Python	Langage utilisé pour programmer l'application.
Google Colab	Environnement d'exécution du notebook.
TensorFlow / Keras	Construction et entraînement du CNN.
Matplotlib / Seaborn	Affichage des images, courbes et matrice de confusion.
Scikit-learn	Calcul des indicateurs d'évaluation.

Table 3.1 – Outils utilisés pour l'application MNIST

3.3 Chargement et visualisation de la base MNIST

La base MNIST contient 60 000 images d'entraînement et 10 000 images de test. Chaque image est en niveaux de gris et possède une taille de 28×28 pixels. Nous avons commencé par charger les données directement depuis Keras, puis nous avons vérifié leurs dimensions.

```
[1] # Chargement des images et des labels de la base MNIST
✓ 0 s # x_train et y_train sont utilisés pour entraîner le modèle
      # x_test et y_test sont utilisés pour tester le modèle après
      l'entraînement
      (x_train, y_train), (x_test, y_test) = mnist.load_data()

      # Vérification des dimensions des données
      print("Images d'entraînement :", x_train.shape)
      print("Labels d'entraînement :", y_train.shape)
      print("Images de test :", x_test.shape)
      print("Labels de test :", y_test.shape)
```

Figure 3.1 – Extrait du notebook : chargement de la base MNIST

Afin de mieux comprendre les données utilisées, nous avons affiché quelques exemples d'images de la base MNIST. La figure suivante montre que les chiffres ont des formes différentes selon l'écriture.

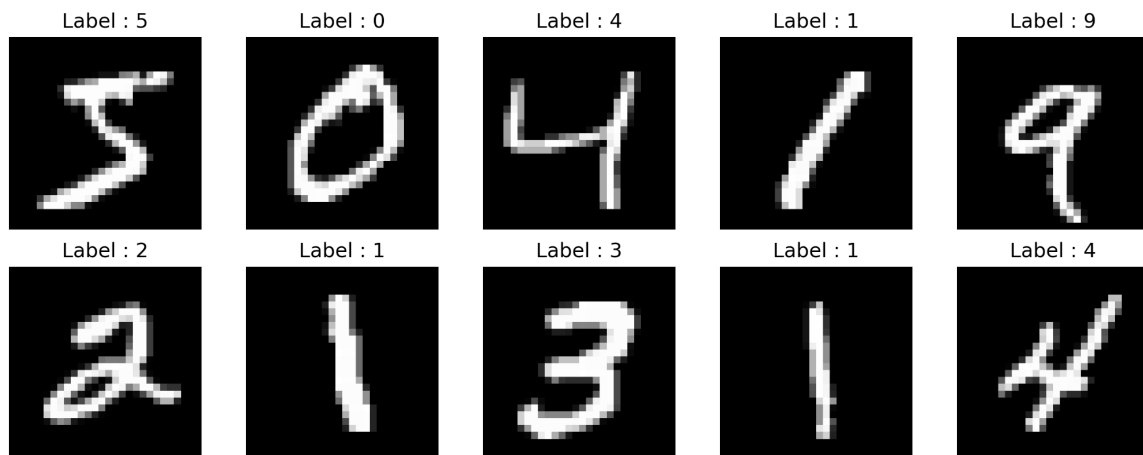


Figure 3.2 – Exemples d'images de la base MNIST

3.4 Prétraitement des données

Avant d'entraîner le modèle, nous avons préparé les données. Les valeurs des pixels sont initialement comprises entre 0 et 255. Nous les avons normalisées entre 0 et 1 afin de faciliter l'apprentissage. Ensuite, nous avons adapté la forme des images au format attendu par les couches convolutionnelles, puis nous avons transformé les labels en vecteurs catégoriels.

```
[2] # Normalisation des pixels entre 0 et 1
✓ 0 s x_train = x_train / 255.0
      x_test = x_test / 255.0

      # Adaptation de la forme des images pour les couches CNN
      # Le dernier 1 correspond au canal de couleur (niveau de gris)
      x_train = x_train.reshape(-1, 28, 28, 1)
      x_test = x_test.reshape(-1, 28, 28, 1)

      # Transformation des labels en vecteurs de taille 10
      y_train_cat = to_categorical(y_train, 10)
      y_test_cat = to_categorical(y_test, 10)
```

Figure 3.3 – Extrait du notebook : prétraitement des images et des labels

Cette étape est importante, car un CNN attend une image sous la forme : nombre d'images, hauteur, largeur et nombre de canaux. Dans notre cas, le canal vaut 1 parce que les images sont en niveaux de gris.

3.5 Architecture du modèle CNN

Nous avons choisi une architecture simple afin de garder une application compréhensible. Le modèle contient deux blocs convolution-pooling, suivis d'une couche Flatten, d'une couche dense, d'une couche Dropout et d'une couche de sortie Softmax. Les couches de convolution permettent d'extraire les caractéristiques des chiffres, tandis que les couches de pooling réduisent la taille des cartes de caractéristiques.

```
[3] # Création du modèle CNN
✓ 0 s model = Sequential()

      # Premier bloc convolution + pooling
      model.add(Conv2D(32, (3, 3), activation="relu", input_shape=(28, 28, 1)))
      model.add(MaxPooling2D((2, 2)))

      # Deuxième bloc convolution + pooling
      model.add(Conv2D(64, (3, 3), activation="relu"))
      model.add(MaxPooling2D((2, 2)))

      # Passage vers les couches de classification
      model.add(Flatten())
      model.add(Dense(128, activation="relu"))
      model.add(Dropout(0.5))

      # Couche de sortie : 10 classes
      model.add(Dense(10, activation="softmax"))
```

Figure 3.4 – Extrait du notebook : construction du modèle CNN

La fonction d'activation ReLU est utilisée dans les couches intermédiaires. La dernière couche utilise Softmax afin d'obtenir une probabilité pour chaque chiffre de 0 à 9.

3.6 Compilation et entraînement

Après la construction du modèle, nous l'avons compilé avec l'optimiseur Adam et la fonction de perte *categorical_crossentropy*. L'entraînement a été réalisé sur 5 époques avec une taille de lot égale à 128. Nous avons aussi réservé 10% des données d'entraînement pour la validation.

```
[4]      # Compilation du modèle
✓ 0 s    model.compile(
          optimizer="adam",
          loss="categorical_crossentropy",
          metrics=["accuracy"]
        )

      # Entraînement du modèle CNN
      history = model.fit(
          x_train,
          y_train_cat,
          epochs=5,
          batch_size=128,
          validation_split=0.1
      )
```

Figure 3.5 – Extrait du notebook : compilation et entraînement du modèle

Les courbes suivantes présentent l'évolution de la précision et de la fonction de perte pendant l'entraînement.

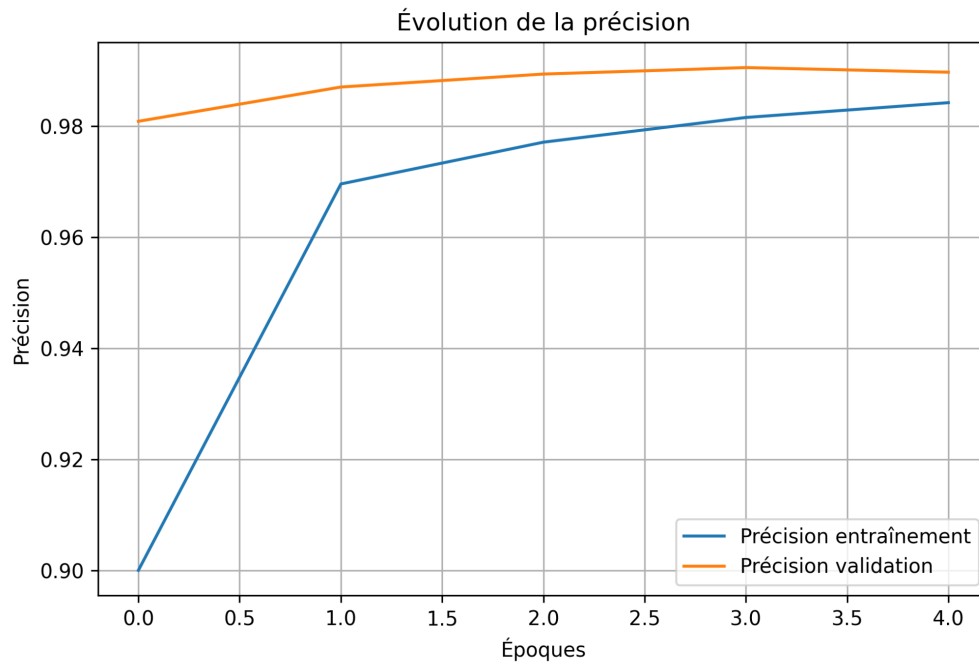


Figure 3.6 – Évolution de la précision pendant l'entraînement

La précision augmente rapidement dès les premières époques. La précision de validation reste élevée et proche de celle de l'entraînement, ce qui montre que le modèle généralise correctement.

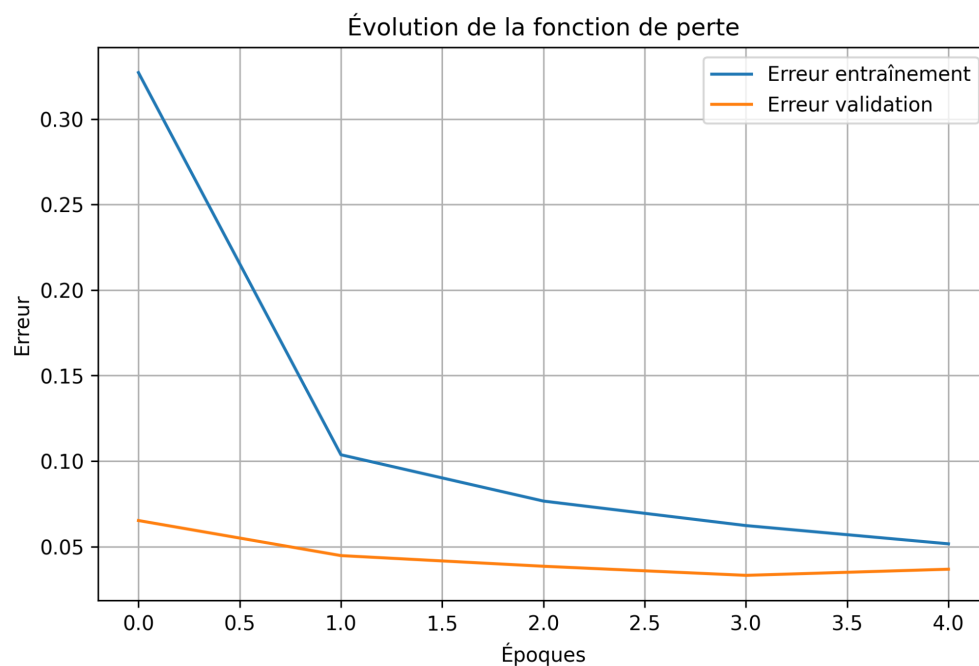


Figure 3.7 – Évolution de la fonction de perte pendant l'entraînement

La fonction de perte diminue progressivement au cours de l'entraînement. Cela indique que le modèle réduit ses erreurs au fil des époques.

3.7 Évaluation et analyse des résultats

Une fois le modèle entraîné, nous l'avons évalué sur les images de test. Ces images n'ont pas été utilisées pendant l'apprentissage, ce qui permet de vérifier la capacité du modèle à reconnaître de nouvelles données.

```
[5] # Évaluation du modèle sur les données de test
✓ 0 s test_loss, test_accuracy = model.evaluate(x_test, y_test_cat)

print("Erreur sur les données de test :", test_loss)
print("Précision sur les données de test :", test_accuracy)

# Prédiction des classes des images de test
y_pred = model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)
```

Figure 3.8 – Extrait du notebook : évaluation du modèle et prédiction des classes

D'après la matrice de confusion obtenue, le modèle classe correctement 9 890 images sur 10 000, soit une précision d'environ 98,90% sur l'ensemble de test.

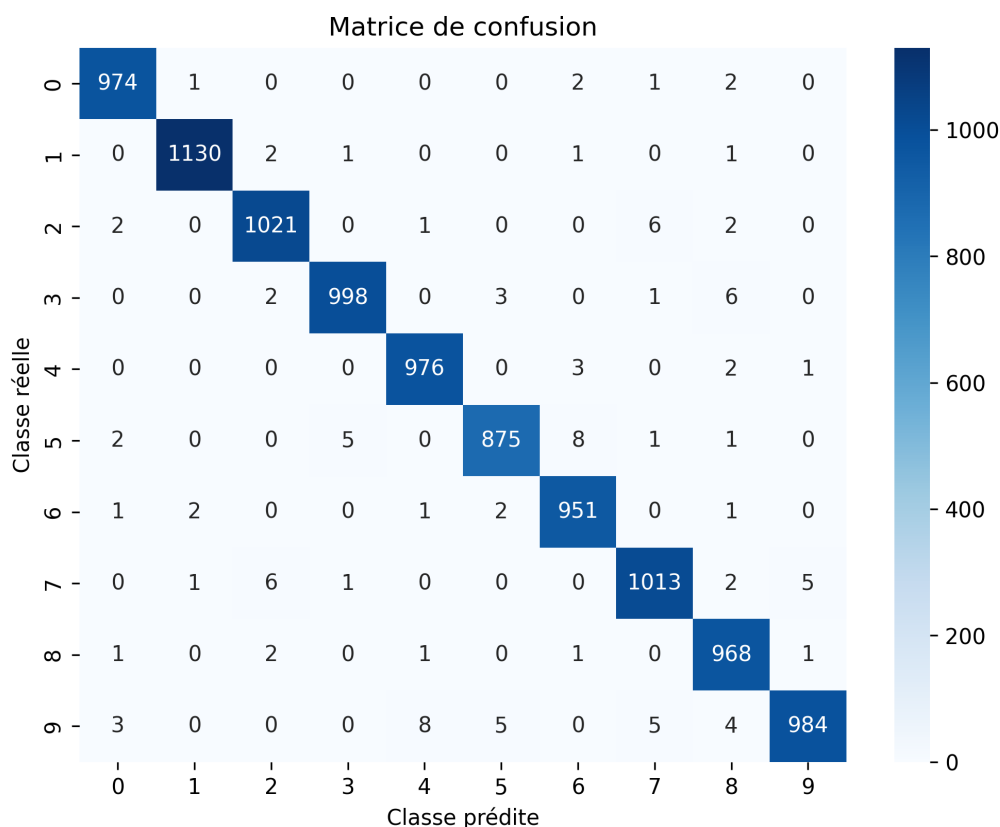


Figure 3.9 – Matrice de confusion du modèle CNN sur MNIST

La matrice de confusion montre que la majorité des images sont correctement classées. Les plus grandes valeurs sont situées sur la diagonale principale. Les erreurs restantes concernent surtout des chiffres visuellement proches selon l'écriture manuscrite.

Nous avons également affiché quelques prédictions afin de vérifier visuellement le comportement du modèle.

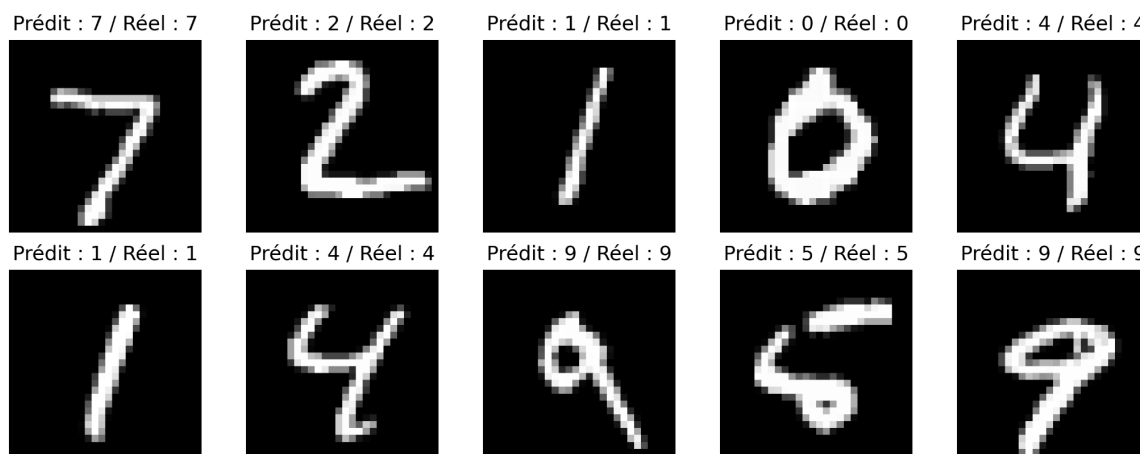


Figure 3.10 – Exemples de prédictions réalisées par le modèle

Ces exemples montrent que le modèle arrive à prédire correctement les chiffres affichés.

3.8 Sauvegarde du modèle

À la fin de l'application, nous avons sauvegardé le modèle afin de pouvoir le réutiliser sans refaire l'entraînement. Nous avons utilisé le format natif de Keras.

```
[6] # Sauvegarde du modèle entraîné au format recommandé par Keras
✓ 0 s model.save("modele_cnn_mnist.keras")
```

Figure 3.11 – Extrait du notebook : sauvegarde du modèle entraîné

3.9 Conclusion

Dans ce chapitre, nous avons réalisé une application pratique d'un réseau de neurones convolutif sur la base MNIST. Nous avons suivi les étapes principales d'un projet de classification d'images : chargement des données, prétraitement, création du modèle, entraînement, évaluation et analyse des résultats.

Les résultats obtenus montrent que le CNN est bien adapté à la reconnaissance de chiffres manuscrits. Les courbes d'apprentissage indiquent une amélioration progressive des performances, tandis que la matrice de confusion confirme que la majorité des chiffres sont correctement classés. Cette application nous a permis de comprendre concrètement l'intérêt des couches de convolution dans l'extraction automatique des caractéristiques d'une image.

Bibliographie

- [1] K. Gurney, *An Introduction to Neural Networks*, CRC Press, 2003.
- [2] Camara94, *base-deep-learning*, dépôt GitHub, 2023. Disponible sur : <https://github.com/camara94/base-deep-learning>.
- [3] C. C. Aggarwal, *Neural Networks and Deep Learning*, Springer, 2018.
- [4] Y. LeCun, L. Bottou, Y. Bengio et P. Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
- [5] J. Castillo Navarro, *Convolutional Neural Networks*, RCP 209, 2025–2026.
- [6] StéphaneDC, *Tutorial CNN partie 1 : MNIST Digits Classification*, Notebook Kaggle, 2020. Disponible sur : <https://www.kaggle.com/code/stephancedc/tutorial-cnn-partie-1-mnist-digits-classification>.